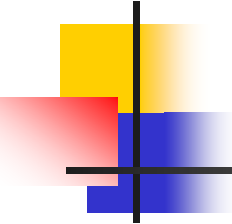


算法设计与分析

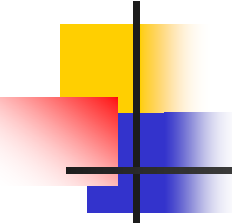
Analysis and Design of Algorithm

Lesson 16



要点回顾

- 第六章 分支限界法（结束）
 - 单源最短路径
 - 最大团问题
- 第七章 随机化算法
 1. 随机数值算法
 - 计算 π 和定积分
 2. Sherwood算法
 - 快速排序
 3. Las Vegas算法
 - N后问题
 4. Monte Carlo算法



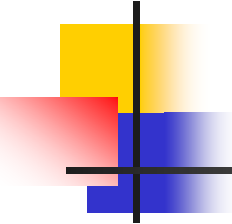
4. Monte Carlo算法

■ 性质：

- 主要用于求解需要准确解的问题
- 算法可能给出错误解
- 获得精确解概率与算法执行时间成正比

■ 应用范例

- 找主元素
- AMS物理实验
- 素数测试



主元素问题

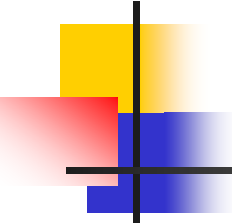
■ 问题定义

- 若 n 元数组 T 中一半以上元素都是 x ，则 x 是 T 的主元素。
- 如何找出数组 T 的主元素

例如： $T[7]=\{3,2,3,2,3,3,5\}$ ，主元素为3

■ 传统解法

- 统计各元素出现的次数，时间复杂度 $O(n^2)$
- 是否还可以改进？



利用Monte Carlo算法求解主元素问题

■ MC算法

- 随机选数组中的元素 $T[i]$ 统计，如果元素出现次数大于 $n/2$ ，即主元素，返回 $true$ ，否则 $false$

■ 成功率分析

- 如果存在主元素在数组中，那么主元素个数大于 $n/2 \rightarrow$ 出错概率小于 $1/2$ 。

算法以大于 $1/2$ 概率返回 $true$;

算法以小于 $1/2$ 概率返回 $false$;

如连续运行MC算法 k 次， $false$ 减少为 2^{-k} ，算法发生错误概率 2^{-k}

```

#include "RandomNumber.h"
#include <iostream>
#include <cmath>
using namespace std;
RandomNumber rnd;
//p正确偏y0蒙特卡洛算法
template <typename Type>
bool Majority(Type *T, int n){
    // 产生0 ~ n-1 之间的随机整数
    int i = rnd.Random(n) + 1;
    Type x = T[i];    // 随机选择数组元素
    int k = 0;
    for(int j = 0; j < n; ++j)
        if(T[j] == x)
            ++k;
    return (k > n/2);
}
// 重复调用算法Majority
template <typename Type>
bool MajorityMC(Type *T, int n, double e){
    int k = ceil(log(1.0/e) / log(2.0));
    for(int i = 1; i <= k; ++i)
        if(Majority(T, n))
            return true;
    return false;
}

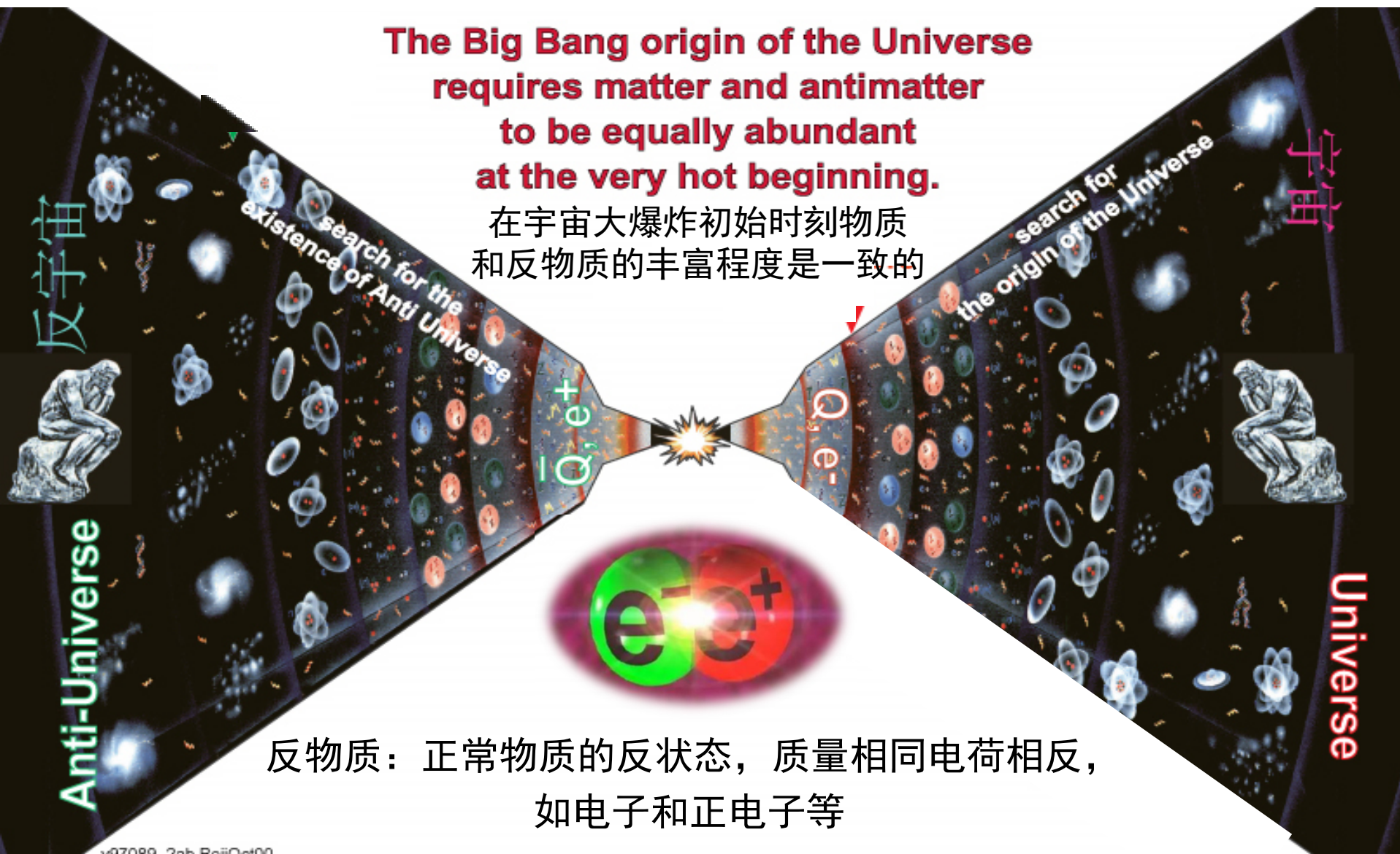
```

对于任何给定的 $e > 0$ ，算法 `majorityMC` 重复调用 $\lceil \log(1/e) \rceil$ 次算法 `majority`。它错误概率小于 e 。算法 `majorityMC` 所需的计算时间显然是 $O(n \log(1/e))$ 。

从AMS实验说起

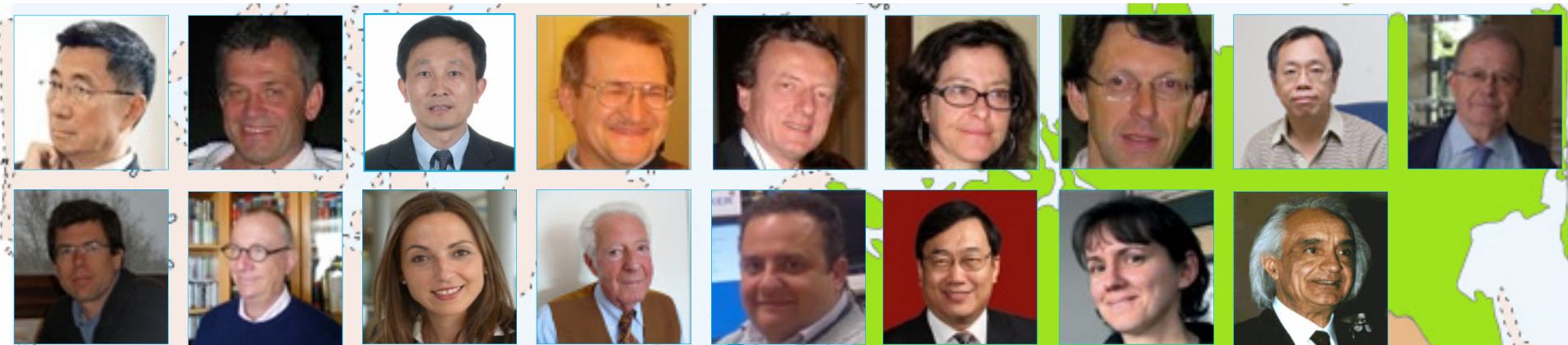
**The Big Bang origin of the Universe
requires matter and antimatter
to be equally abundant
at the very hot beginning.**

在宇宙大爆炸初始时刻物质
和反物质的丰富程度是一致的



反物质：正常物质的反状态，质量相同电荷相反，
如电子和正电子等

AMS实验是由诺贝尔奖获得者丁肇中教授领导，由美国、中国（台湾）、俄罗斯、法国、德国、意大利、瑞士、芬兰等15个国家在空间领域的大合作，包括60个大学和研究所，600位科学家。



USA

A&M FLORIDA UNIV.
JOHNS HOPKINS UNIV.
MIT - CAMBRIDGE
NASA GODDARD SPACE FLIGHT CENTER
NASA JOHNSON SPACE CENTER
UNIV. OF MARYLAND-DEPT OF PHYSICS
UNIV. OF MARYLAND-E.W.S. S.CENTER
YALE UNIV. - NEW HAVEN

MEXICO

UNAM

DENMARK

UNIV. OF AARHUS

FINLAND

HELSINKI UNIV.
UNIV. OF TURKU

FRANCE

GAM MONTPELLIER
LAPP ANNECY
LPSC GRENOBLE

GERMANY

RWTH-I
RWTH-III
MAX-PLANK INST.
UNIV. OF KARLSRUHE

ITALY

ASI
CARSO TRIESTE
IROE FLORENCE
INFN & UNIV. OF BOLOGNA
INFN & UNIV. OF MILANO
INFN & UNIV. OF PERUGIA
INFN & UNIV. OF PISA
INFN & UNIV. OF ROMA
INFN & UNIV. OF SIENA

NETHERLANDS

ESA-ESTEC
NIKHEF
NLR

PORTUGAL

LAB. OF INSTRUM. LISBON

ROMANIA

ISS
UNIV. OF BUCHAREST

RUSSIA

I.K.I.
ITEP
KURCHATOV INST.
MOSCOW STATE UNIV.

SPAIN

CIEMAT - MADRID
I.A.C. CANARIAS.

SWITZERLAND

ETH-ZURICH
UNIV. OF GENEVA

中国

中科院电工所
中科院高能所
东南大学
上海交大
中山大学
山东大学
航天部一院
航天部五院

中山科学院
中央研究院
中央大学
成功大学
太空计划室
交通大学

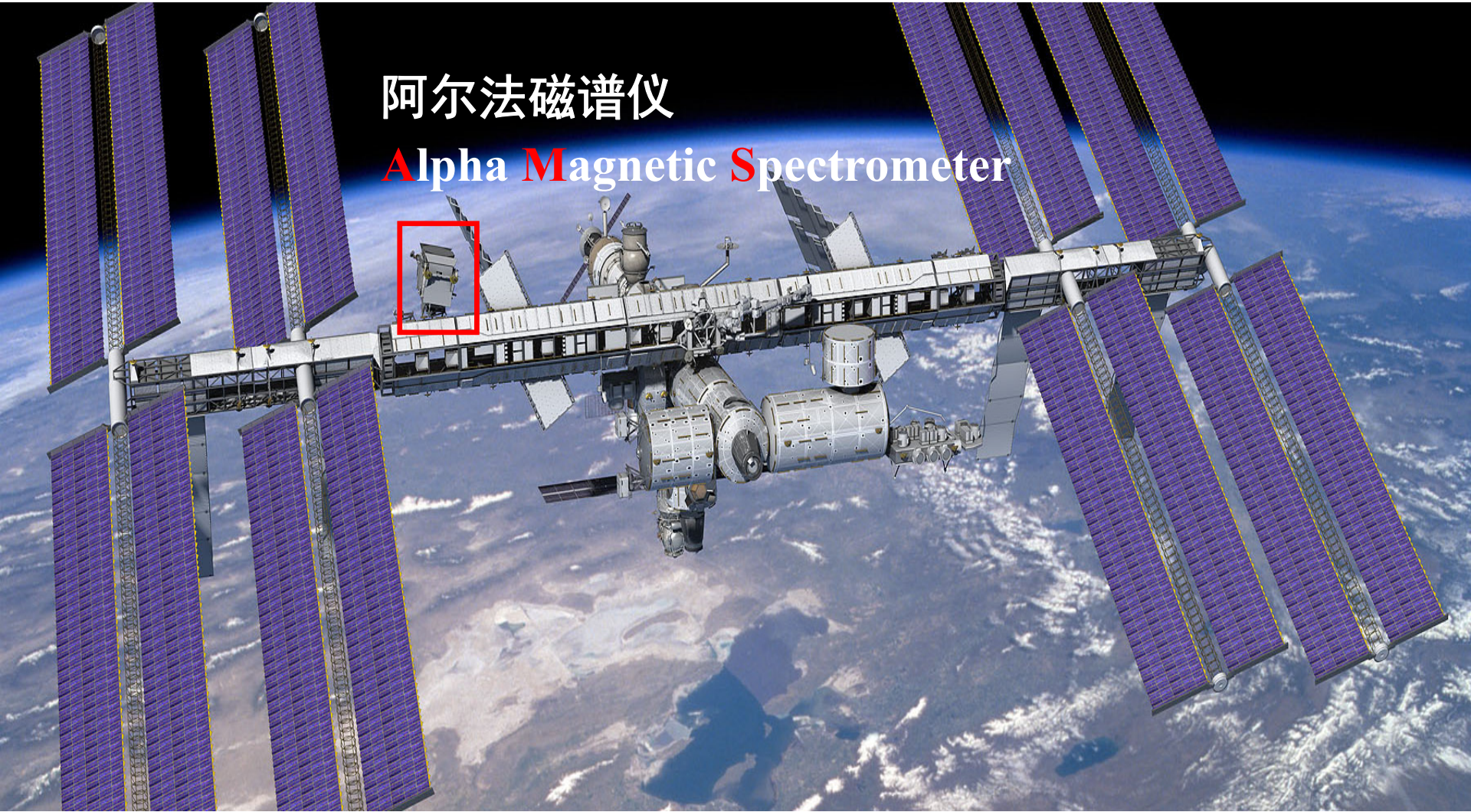
KOREA

EWHA
KYUNGPOOK NAT.UNIV.

AMS是唯一一个直接在国际空间站中探测反物质和暗物质的大型科学实验

阿尔法磁谱仪

Alpha Magnetic Spectrometer



5m x 4m x 3m

7.5 tons

300,000 电子信号通道(electronic channels)

650 微处理器(microprocessors)

穿越辐射探测器
(TRD)

AMS

硅微条
(Silicon layer)

飞行计时器
(TOF 1, 2)

磁铁(Magnet)

7层硅微条
(7 Silicon layers)

飞行计时器
(TOF 3, 4)

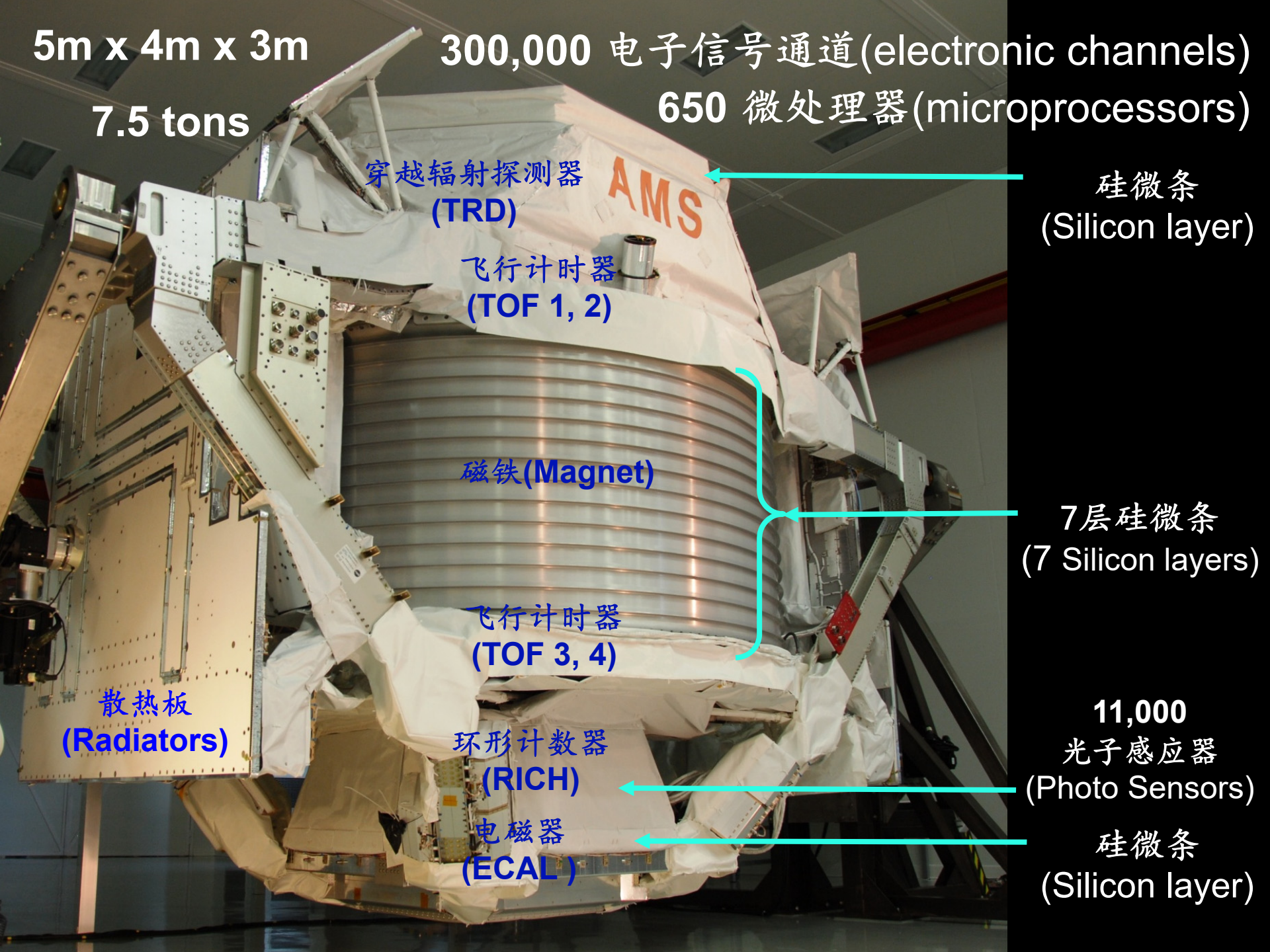
散热板
(Radiators)

环形计数器
(RICH)

11,000
光子感应器
(Photo Sensors)

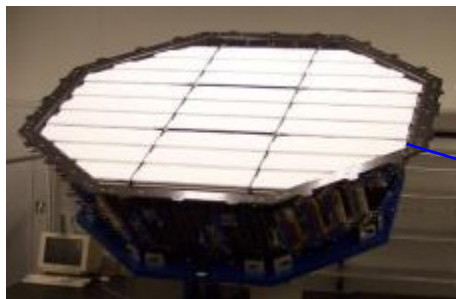
电磁器
(ECAL)

硅微条
(Silicon layer)



AMS: 万亿电子伏 精密的磁谱仪

穿越辐射 探测器
识别 e^+ , e^-



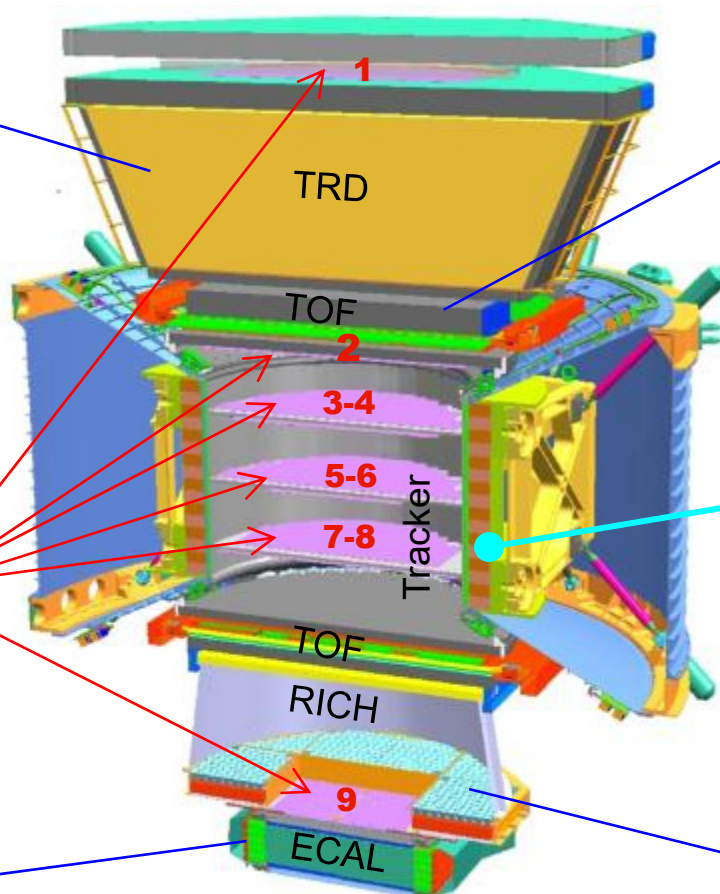
硅微条探测器
 Z , P



电磁量能器
 e^+ , e^- 和光子的能量



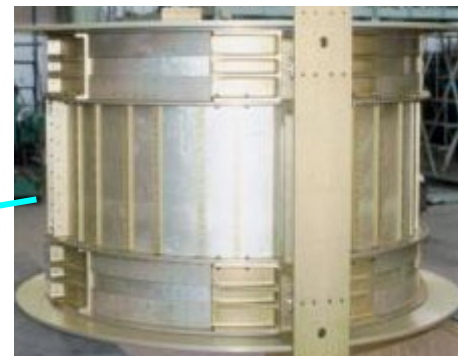
粒子通过它们的电荷 (Z) 和能量
($E \sim P$) 来确定



飞行时间计数器
 Z , E



永磁体
 $\pm Z$



切伦科夫 探测器
 Z , E

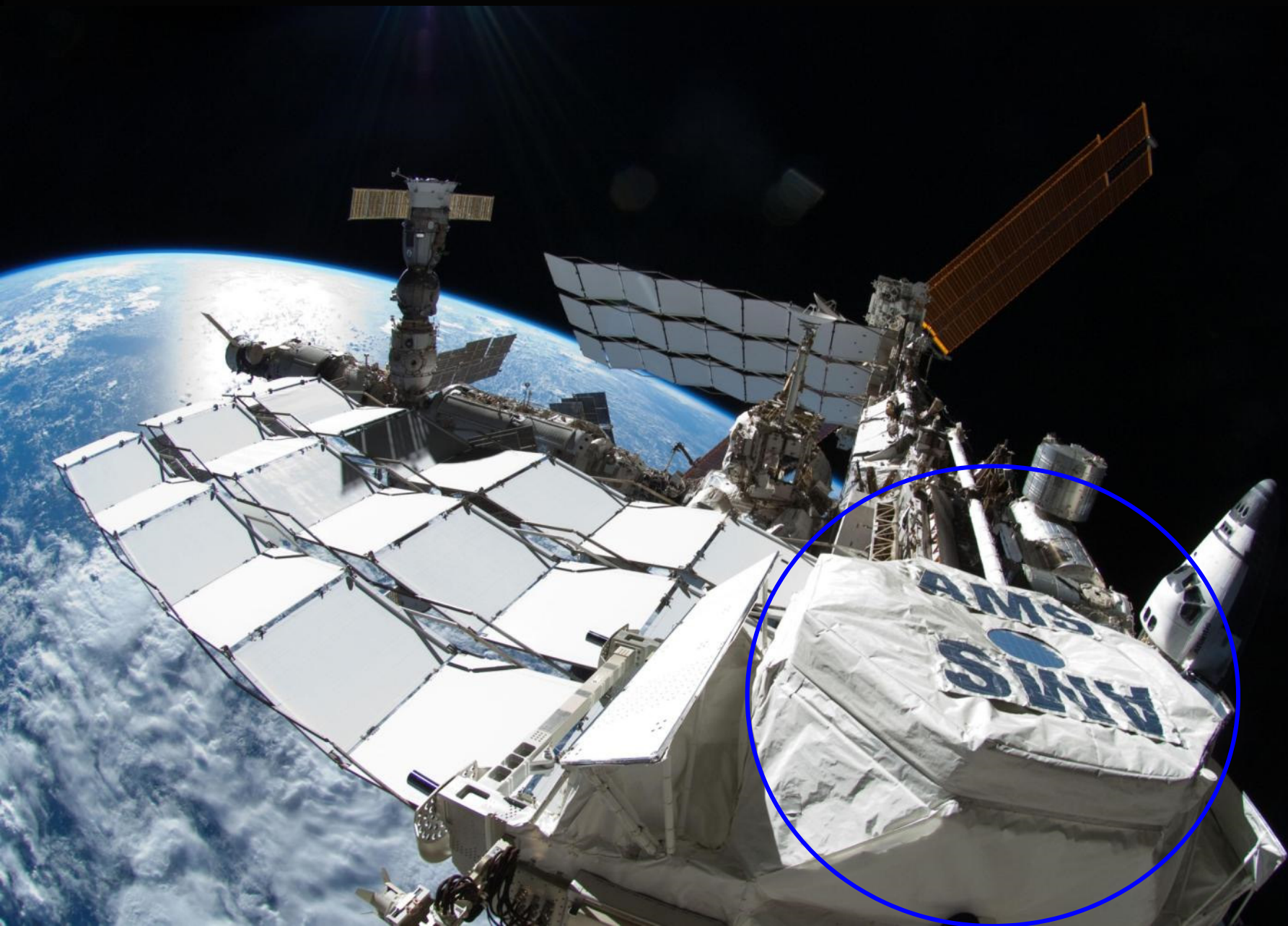


Z , P 被硅微条 探测器, 切伦科夫探
测器, 飞行时间计数器 和电磁量能
器 相互独立地测量



2011年5月16号，搭载AMS的奋进号航天飞机（STS-134）发射

2011年 5月19日 5:15 AMS安装完毕， 9:35开始接收数据







DATA&Lead

PM

TEE

Thermal

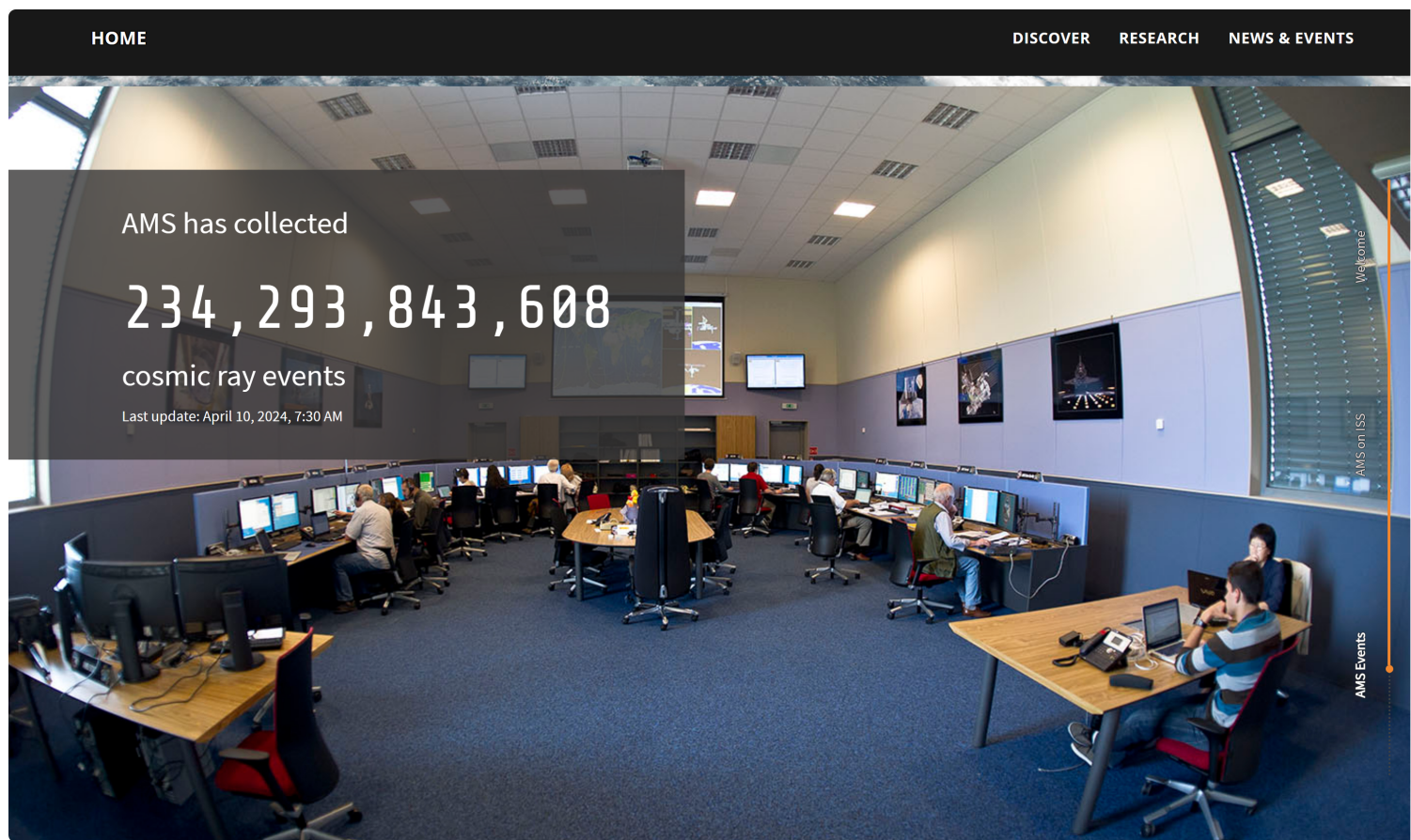
SOC

AMS科学家 必须随时与美国宇航局空间站控制中心协调



从AMS实验说起

- 自2011年5月19号始，已采集数以千亿次的粒子事件，Peta量级的总数据量





东南大学**AMS-SOC**数据处理中心投资**2500万**，主要完成**AMS**原始数据存储管理、蒙特卡洛仿真、数据重建以及物理分析等任务

MIT的丁肇中教授和Vitaly Choutko教授访问东南大学云计算中心，共同商讨东南大学AMS-SOC的设计和建设



丁教授明确指出：东南大学**AMS-SOC**需要承担包括蒙特卡罗仿真计算、原始数据的传输和备份、数据重建以及分发等核心任务，并由**Vitaly Choutko**全面负责与东南大学开展**AMS-SOC**相关工作。



丁肇中教授与易红书记和罗军舟教授
商讨东南大学**AMS-SOC**的下一步工作计划



计算机科学与工程学院

School of Computer Science and Engineering



2016年3月3日，丁肇中教授夫妇访问
东南大学计算机科学与工程学院**AMS**团队

东南大学



1. AMS-C地面控制系统

2. AMS-AIS反物质模拟探测系统

3. AMS-SOC数据处理分析中心

■ AMS实验与**随机化算法**（蒙特卡罗算法）

- 生成海量仿真数据
- 比对宇宙中采集的原始数据
- 证明反物质的存在

■ 最新进展

- 完成了大规模的AMS数据仿真并产生约1600TB的仿真数据
- 所有的AMS原始数据约400TB
- 已证明反物质存在的六大前提条件中的五个

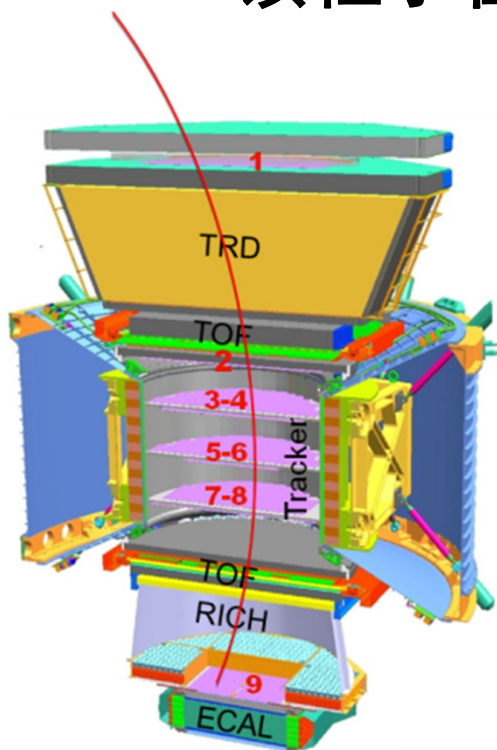
AMS实验中的蒙特卡洛方法

1. 实验设计中蒙特卡洛方法的应用

- 例如要确定带电粒子的动量，通常可以从测量该粒子在磁场中径迹的曲率来得到。

$$p = 3 \times 10^{-2} B Z \rho (\text{GeV} / c)$$

其中 p 为粒子动量， Z 为该粒子电荷（以电子电荷为单位）。 B 为磁场强度，用KGS为单位。 ρ 为径迹曲率，以m（米）为单位



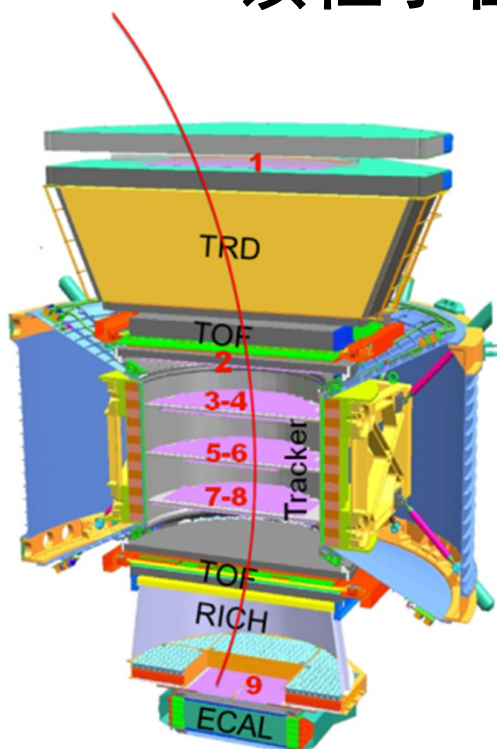
AMS实验中的蒙特卡洛方法

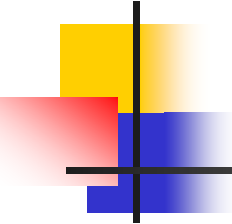
1. 实验设计中蒙特卡洛方法的应用

- 例如要确定带电粒子的动量，通常可以从测量该粒子在磁场中径迹的曲率来得到。

$$p = 3 \times 10^{-2} B Z \rho (\text{GeV} / c)$$

该曲率是通过沿径迹取很多点的坐标测量值计算出来的。这样计算出的动量实际上包含了探测器对径迹空间的有限分辨率引起的误差，还包括了粒子穿过探测器时，在其中各种材料上多次散射造成的误差。这些效应具有随机性。它们可以直接用蒙特卡洛的计算方法来确定这些效应的数值。

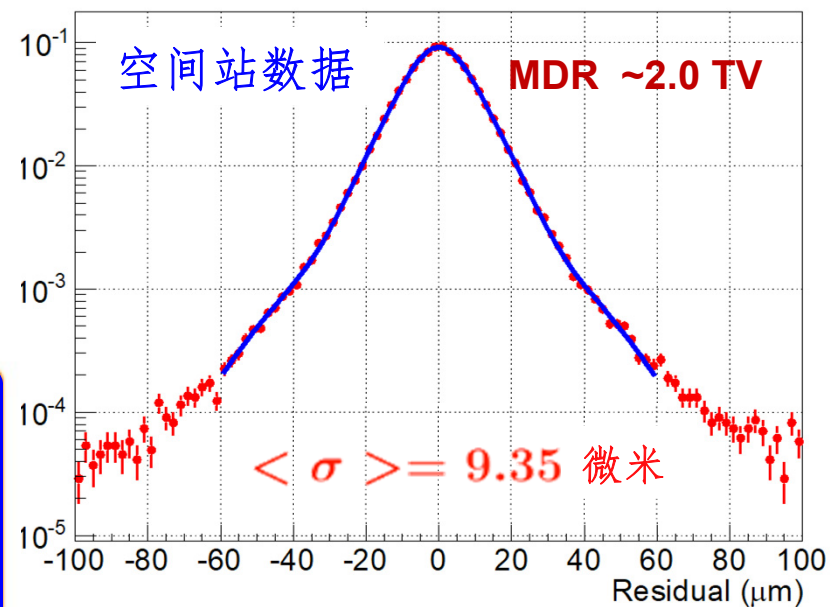
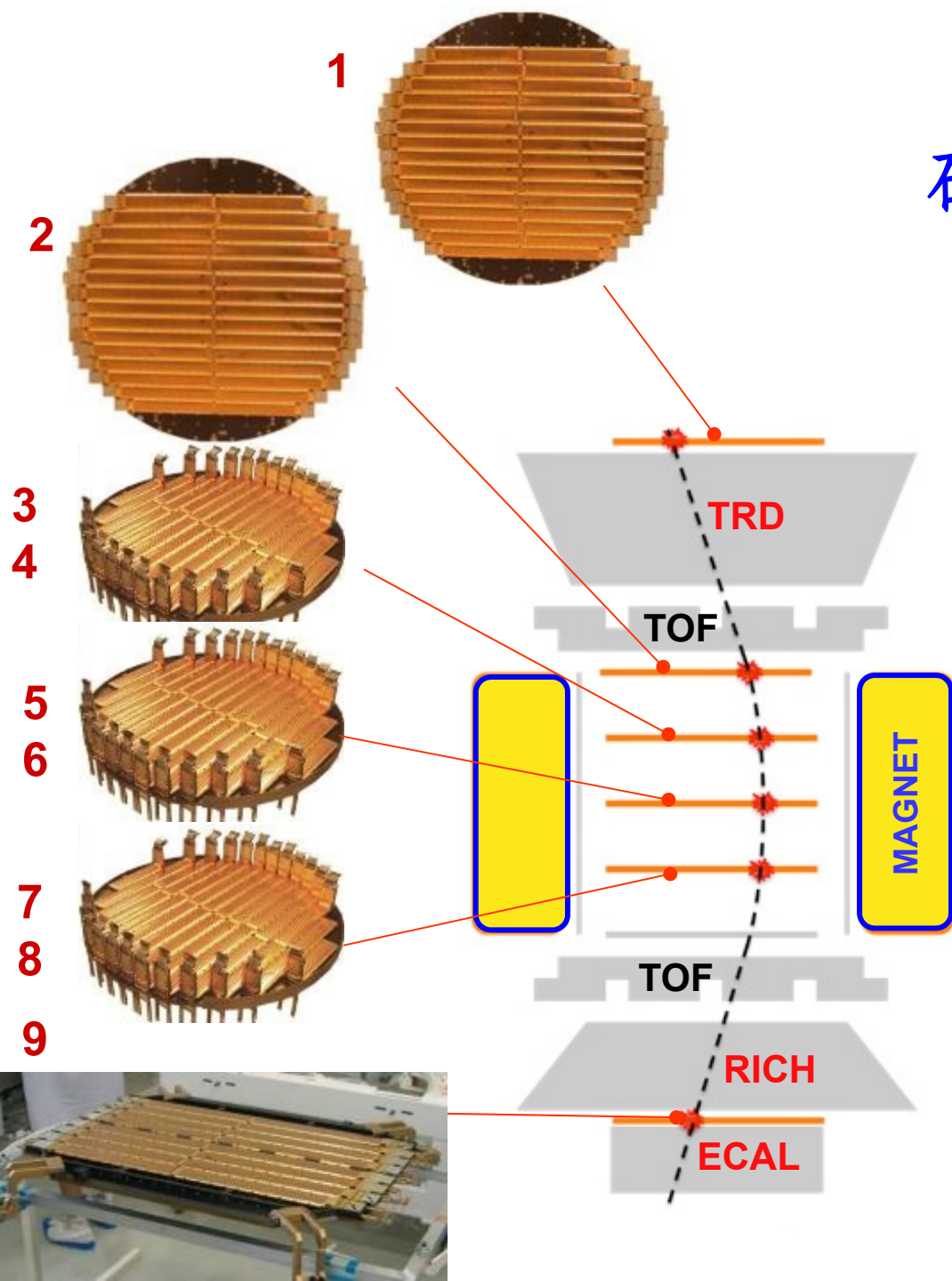


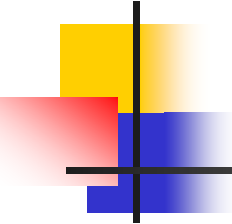


AMS实验中的蒙特卡洛方法

- 一般情况下，模拟计算得到的动量分辨率是粒子动量的函数。但是如果模拟某个探测装置的动量分辨率值很大，则探测装置设计就应当做修改。例如：提高磁场强度、重新安排探测器以测量更多的空间坐标参数、改进探测器位置测量精度、或者减小该装置中材料的密度等等。
- 实际上，在对实验装置进行设计的阶段，需要对探测器做大量的类似上面介绍的模拟研究，以了解该装置中各个探测器的响应，并进一步判断该装置是否能满足各项指标的要求以及探测器的安排和设计是否合理。

硅微条探测器

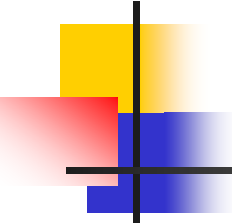




AMS实验中的蒙特卡洛方法

2. 数据分析中蒙特卡洛模拟方法的应用

- 在高能物理实验中，常常用一些大型、复杂的程序来分析实验数据和对实验数据进行筛选分类。
- 为了检验这些程序的可靠性，可以采用输入一些已知数据格式的蒙特卡洛数据，以检验该程序**能否总是成功地重建输入数据**。
- 这种方法非常有用。特别在实验装置运行之前，采用蒙特卡洛模拟数据来检验程序就更为必要。



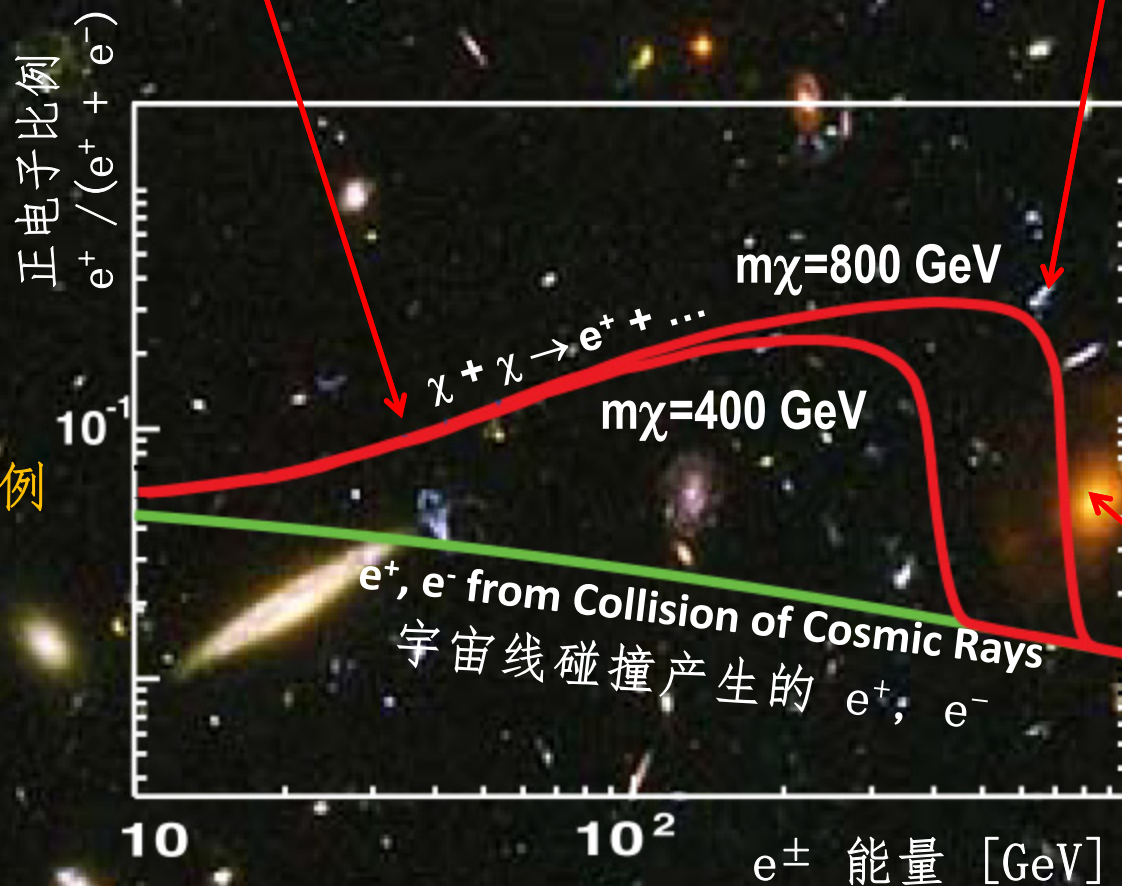
AMS实验中的蒙特卡洛方法

2. 数据分析中蒙特卡洛模拟方法的应用

- 在粒子物理实验中，蒙特卡洛程序可以根据过程的理论规律，产生出主过程和本底过程事例，由此给出末态粒子的所有径迹参数。然后再将这些径迹参数输入到分析程序中就可以算出该装置的探测效率和本底过程对全截面测量的影响。

2. 能量增加时的上升率
3. 是否存在明显的结构

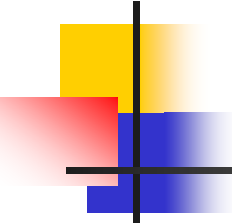
4. Turning Point
到什么能量停止上升



5. Isotropy
各向同性

1. 正电子比例
开始上升
的能量点

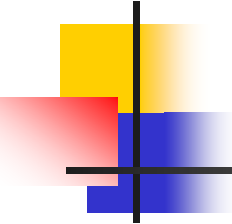
6. 转折点之后
下降的速度



AMS实验中的蒙特卡洛方法

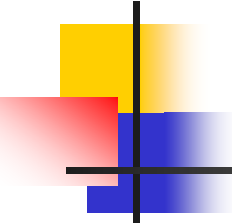
■ 小结：

- 实验进行前，可用MC模拟所要研究事例的特性；
- 在**探测器的设计**过程中，可利用MC模拟的结果来**优化**探测器对所要研究的物理过程的**探测效率**：探测器的构成、各部件的尺寸及相互位置等；
- 在**实验数据分析**过程中，可用MC模拟产生的数据来**调整**分析程序中算法和**参数**，以得到感兴趣的物理信号；
- 可用MC模拟方法来估计探测器的接受度和探测效率对实验数据的修正，使得试验结果可与理论预言相比较；通过与实验结果进行比较可检验模拟程序中所用的理论模型的正确性。



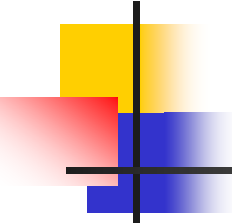
第七章小结

- 理解随机数、伪随机数
- 掌握数值随机算法的设计思想
 - 主要用于数值问题求解，往往是近似解
 - 近似解的精确度与算法执行时间成正比
 - 应用范例（ π ，积分，非线性方程）
- 掌握Sherwood算法的设计思想
 - 一定能够求得一个正确解
 - 算法最坏与平均复杂性差别大时，加随机性
 - 消除最坏行为与特定实例的联系
 - 应用范例（优化快速排序）
- 掌握Las Vegas算法的设计思想
 - 反复运行LV算法，直到得到正确解为止
 - n 皇后问题



第七章小结

- 掌握蒙特卡洛算法的设计思想
 - 主要用于求解需要准确解的问题
 - 算法可能给出错误解
 - 获得精确解概率与算法执行时间成正比
 - 应用范例（找主元素、AMS物理实验、素数测试）



课程内容

NP完全性理论与近似算法

算法高级理论

随机化算法

线性规划与网络流

高级算法

递归
分治

动态
规划

贪心
算法

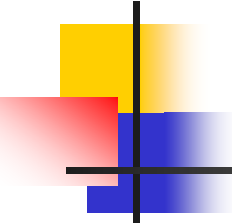
回溯与
分支限界

基础算法

算法分析与问题的计算复杂性

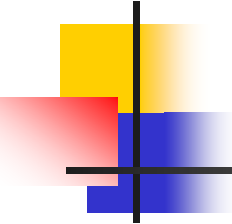
算法基础理论

第八章 线性规划与网络流



学习要点

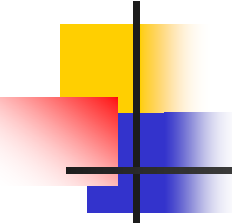
- 理解线性规划模型
- 理解网络与网络流的基本概念
- 掌握网络最大流的增广路径算法



线性规划

■ 线性规划

- 求线性目标函数在线性约束条件下的最大值或最小值的问题
- 满足线性约束条件的解叫做可行解
- 由所有可行解组成的集合叫做可行域



线性规划的形式化定义

$$\max \quad c_1x_1 + c_2x_2 + \cdots + c_nx_n$$

$$\text{s.t.} \quad \sum_{t=1}^n a_{it}x_t \leq b_i \quad (i = 1, 2, \cdots, m_1)$$

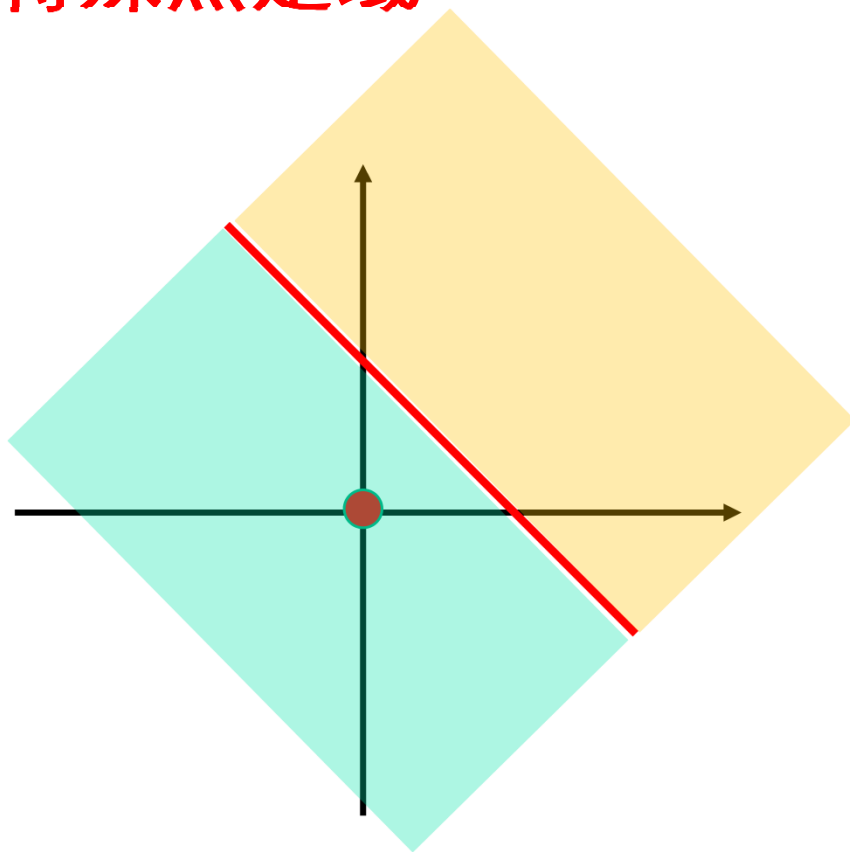
$$\sum_{t=1}^n a_{jt}x_t = b_j \quad (j = m_1 + 1, \cdots, m_1 + m_2)$$

$$\sum_{t=1}^n a_{kt}x_t \geq b_k \quad (k = m_1 + m_2 + 1, \cdots, m_1 + m_2 + m_3)$$

$$x_t \geq 0 \quad (t = 1, 2, \cdots, n)$$

线性规划的求解算法

■ 特殊点定域



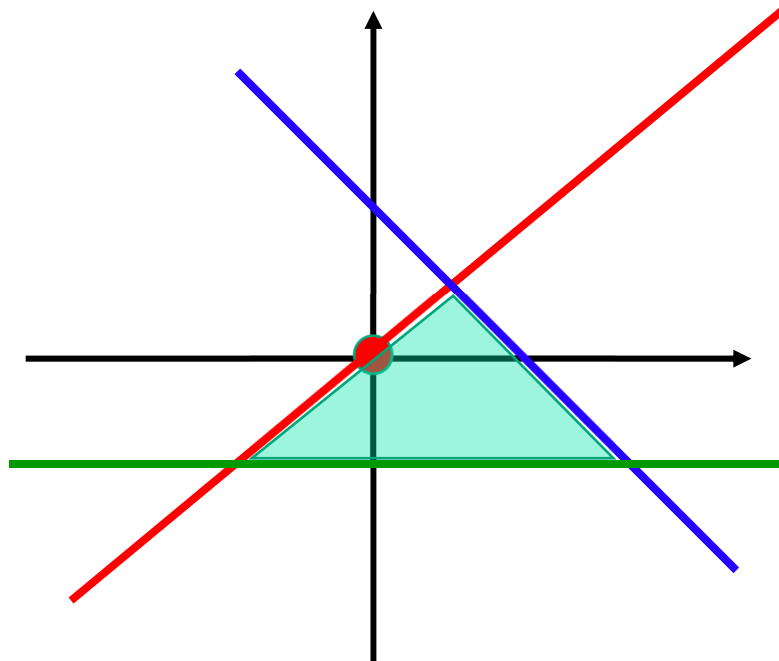
$$x + y - 1 = 0$$

$$x + y - 1 < 0$$

$$x + y - 1 > 0$$

线性规划的求解算法

- 请画出二元一次不等式组所表示的平面域



$$\begin{cases} y \leq x \\ x + y \leq 1 \\ y \geq -1 \end{cases}$$

直线定界、特殊点定域

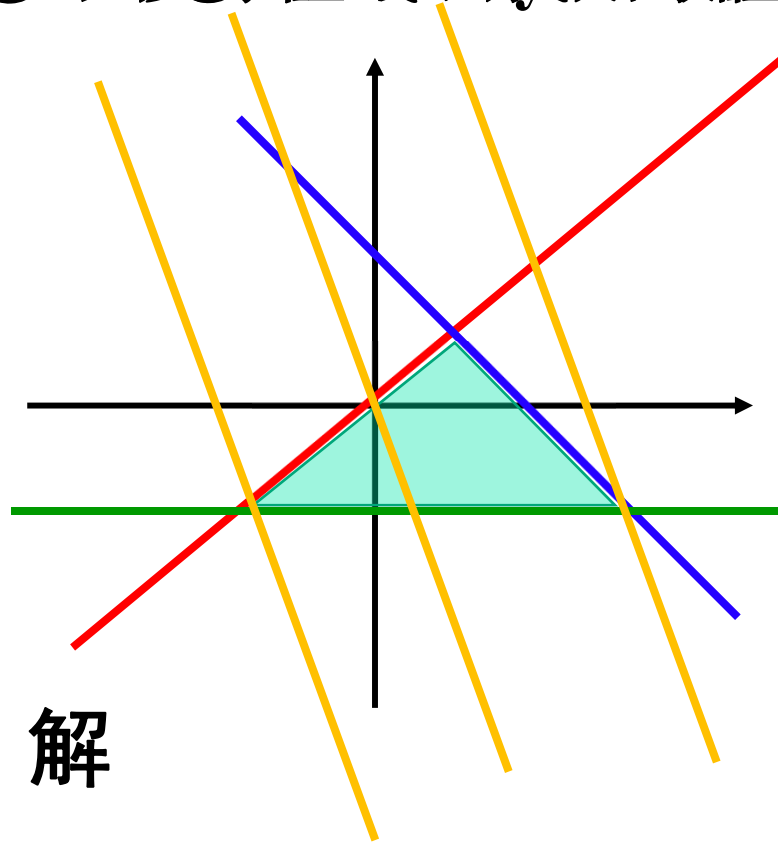
线性规划的求解算法

- 请求出 $Z=2x+y$ 的最大值，其中 x, y 满足如下的二元一次不等式组。

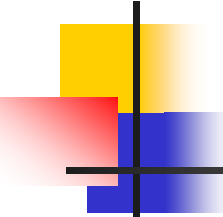
$$\max 2x+y$$

$$s.t \begin{cases} y \leq x \\ x+y \leq 1 \\ y \geq -1 \end{cases}$$

③ 平移②画出的可行域，得最优值线



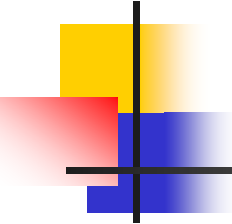
解法：画、移、找、解



线性规划的实际应用

- **例子：**茶颜悦色有两种饮料非常畅销。已知甲饮料每杯含奶4克，糖3克；乙饮料则是每杯含奶5克，糖10克。考虑成本约束，这家店每天只准备了2000克的奶粉和3000克的糖。问：
 - 该店每天调制两种饮料的杯数可能是多少？
 - 如果甲饮料每杯能获利1元，乙饮料每杯能获利1.5元，假设供不应求，那么如何配置这两种饮料的杯数，使得获利最大？

解法：画、移、找、解



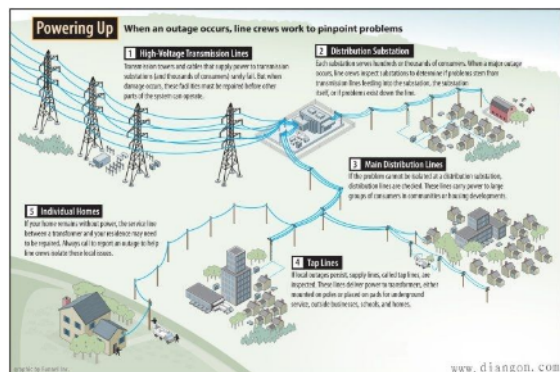
线性规划的求解算法

■ 单纯形法

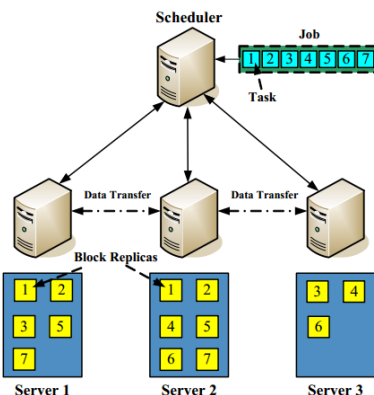
- 先找出一个基本可行解，看是否是最优解
 - 若不是，则按照一定法则转换到另一改进后更优的基本可行解，再鉴别
 - 若仍不是，则再转换，按此重复进行
-
- 因基本可行解的个数有限，故经有限次转换必能得出问题的最优解

- **网络流(network flows)**
 - 一种类比水流的解决问题方法

■ 网络流的应用



电力网络中的 电力输送问题



基于二分匹配的任务调度问题

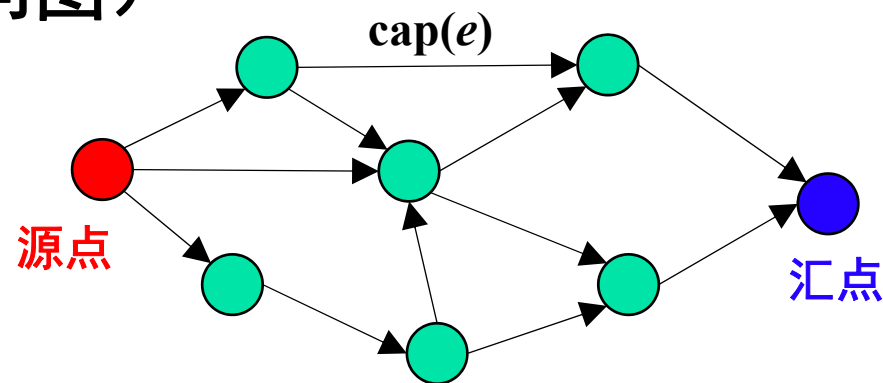


交通网络中的 车流调度问题

网络流—最大流问题

■ 网络 $G=(V,E)$ 的定义（有向图）

- 有源点 s 和汇点 t
- 每条边 e 有容量 $\text{cap}(e) \geq 0$

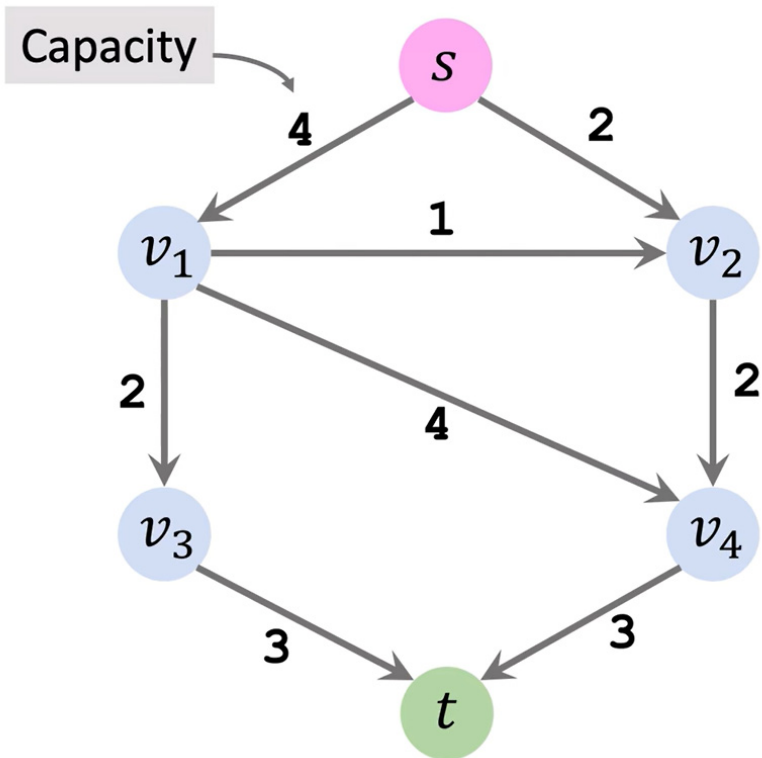


■ 网络流的定义

- 边集合 E 上的一个非负函数 $\text{flow}=\{\text{flow}(e)\}$ ，称为流量
- 边 e 上的流量 $\text{flow}(e) \leq \text{边}e\text{容量}\text{cap}(e)$
- 顶点 v 的入边流量和=出边流量和

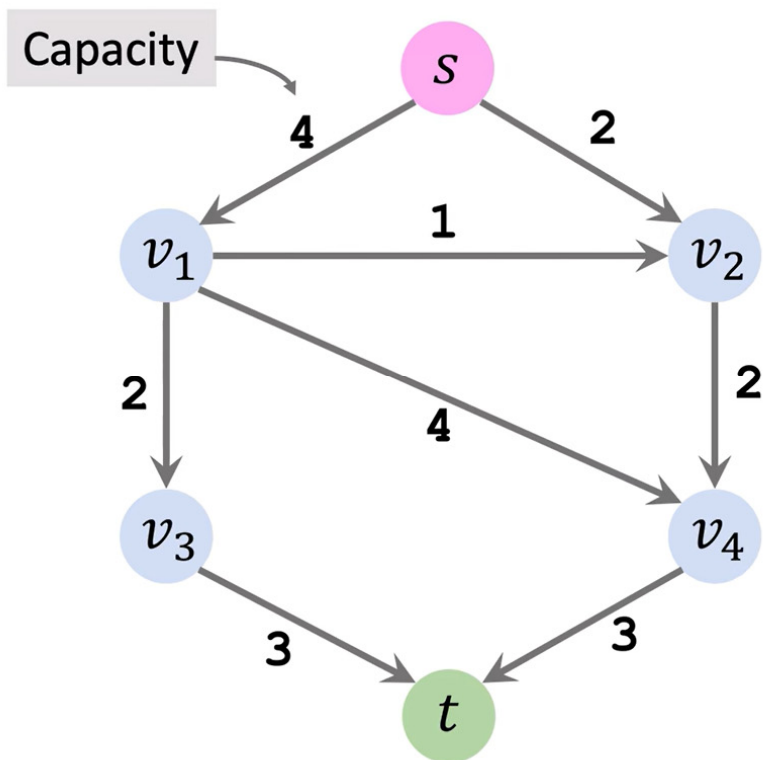
最大流问题： 求汇点入边流量和的最大值

网络流问题的求解算法

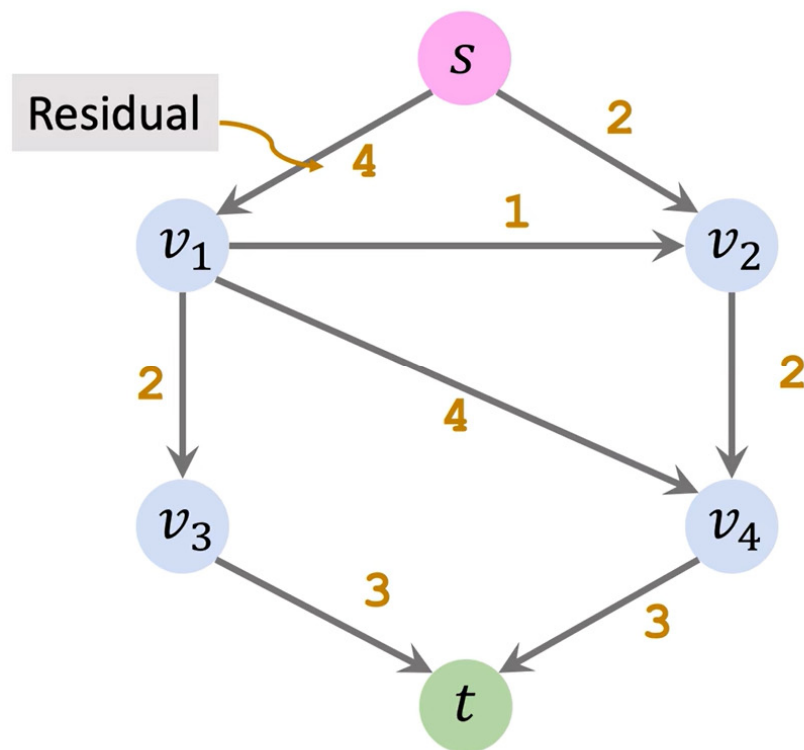


网络流问题的求解算法

方法1: Naive Algorithm



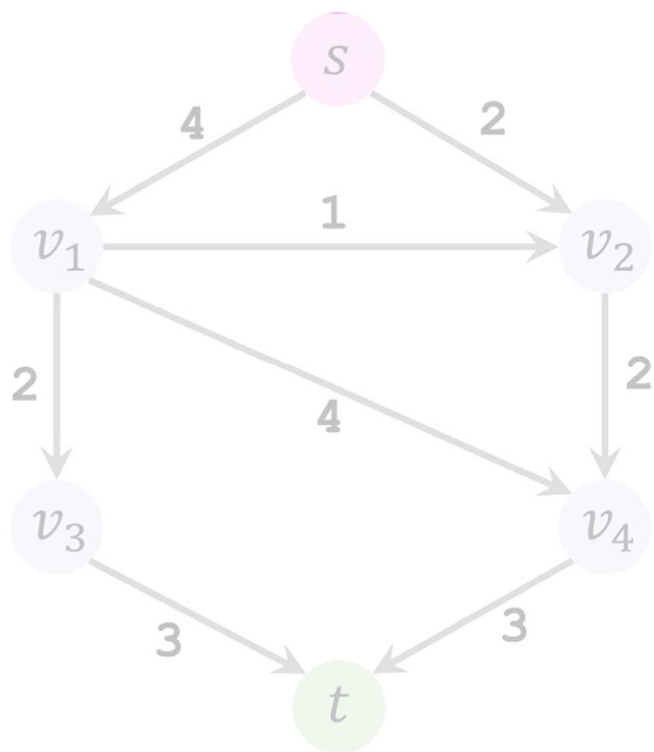
原始网络



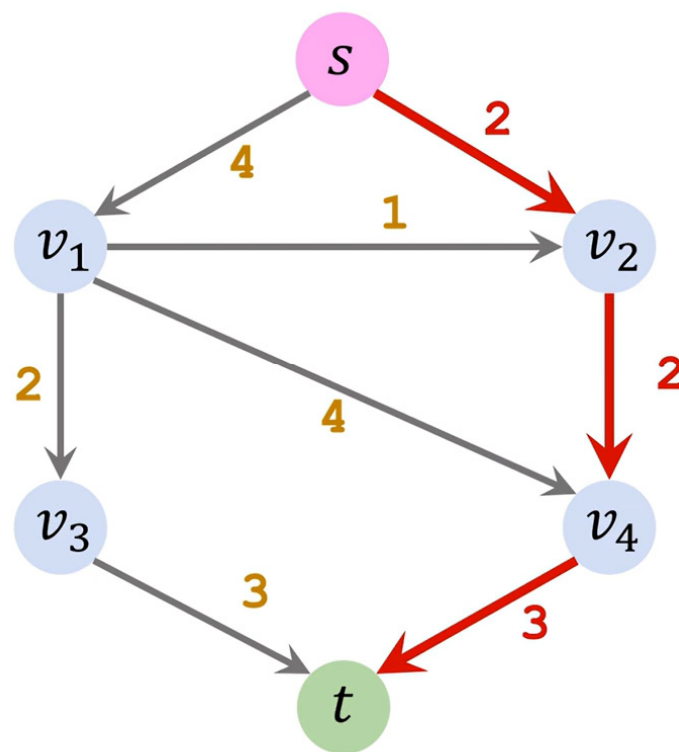
残流网络

网络流问题的求解算法

方法1: Naive Algorithm



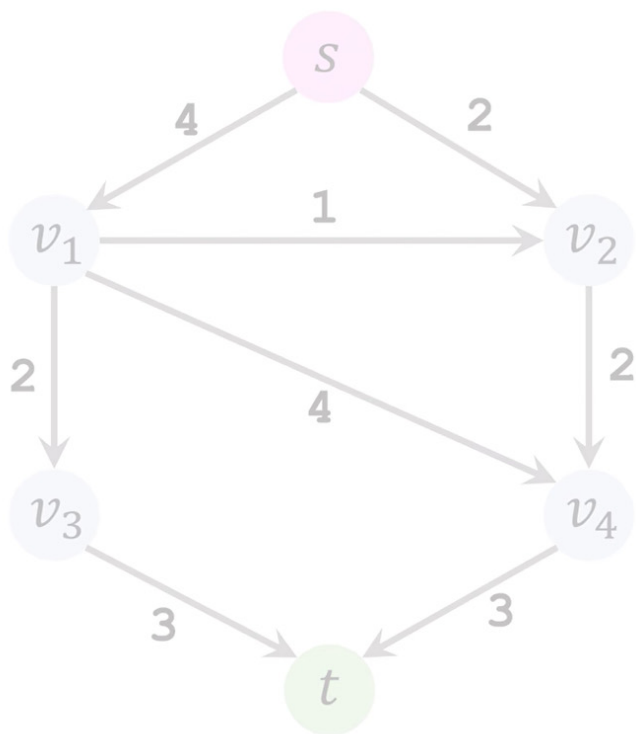
原始网络



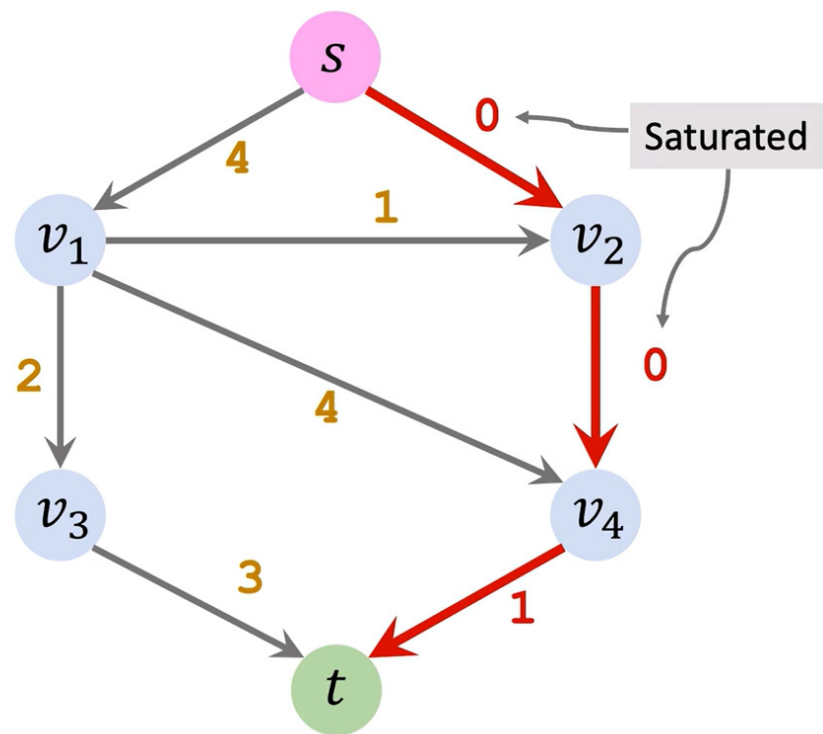
残流网络

网络流问题的求解算法

方法1: Naive Algorithm



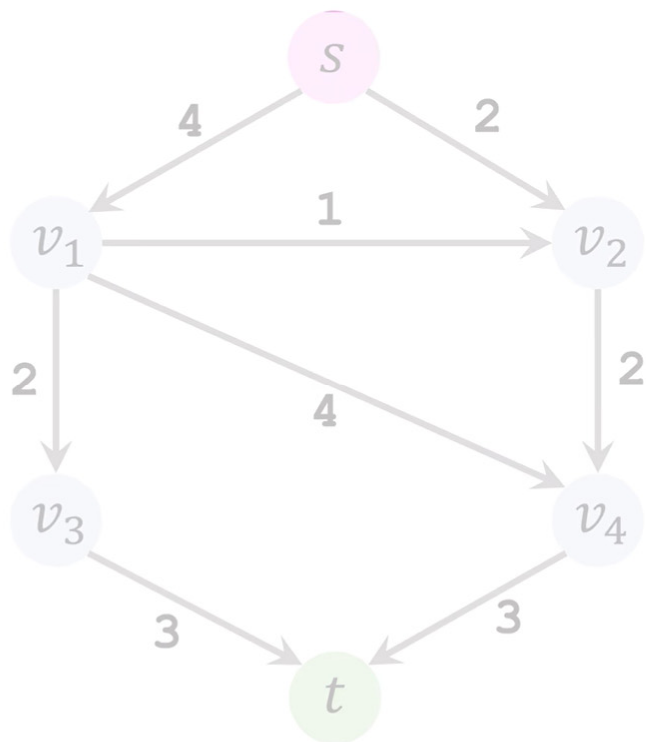
原始网络



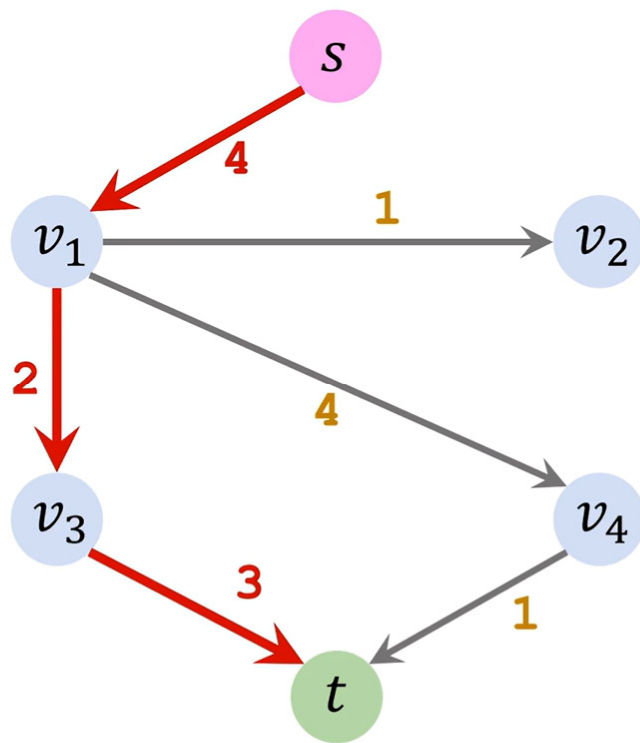
残流网络

网络流问题的求解算法

方法1: Naive Algorithm



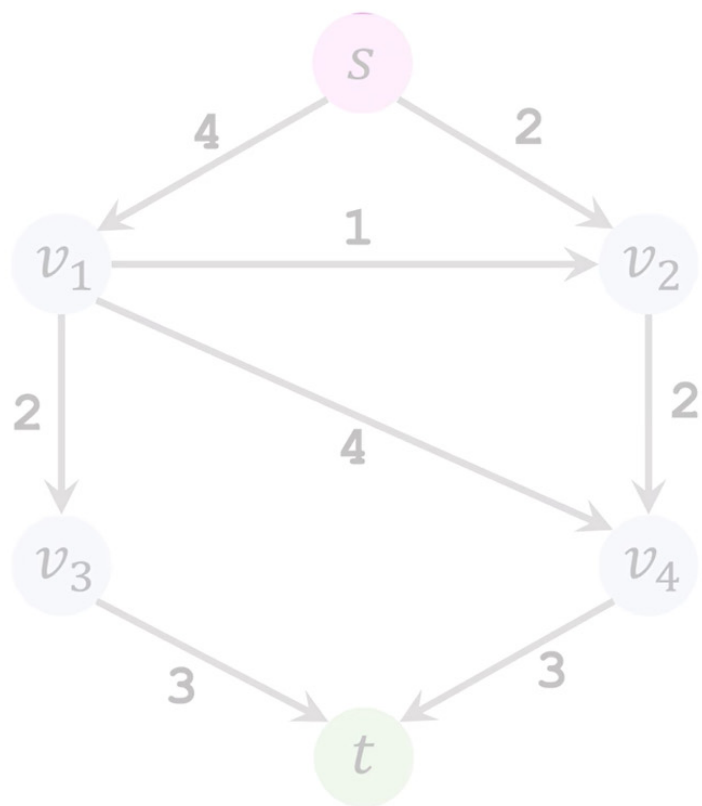
原始网络



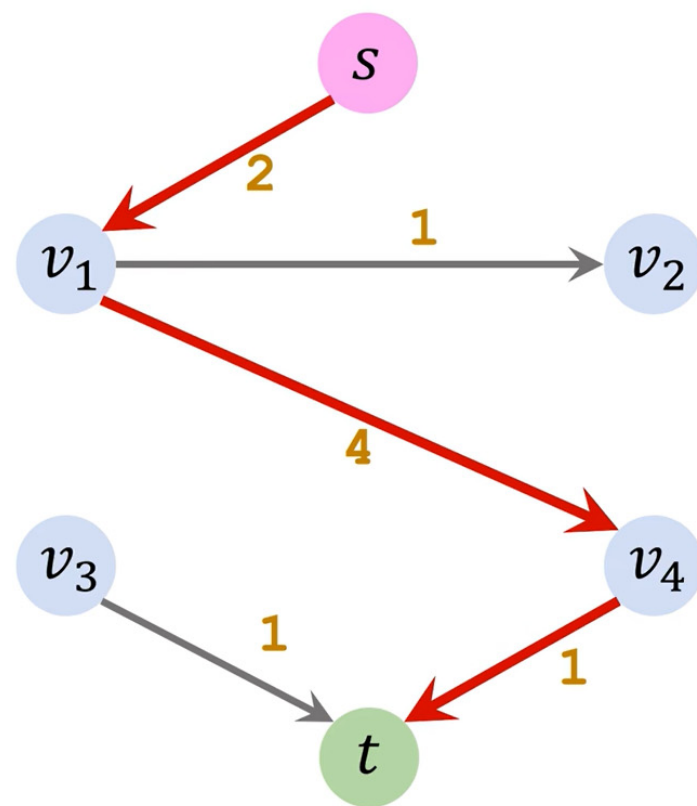
残流网络

网络流问题的求解算法

方法1: Naive Algorithm



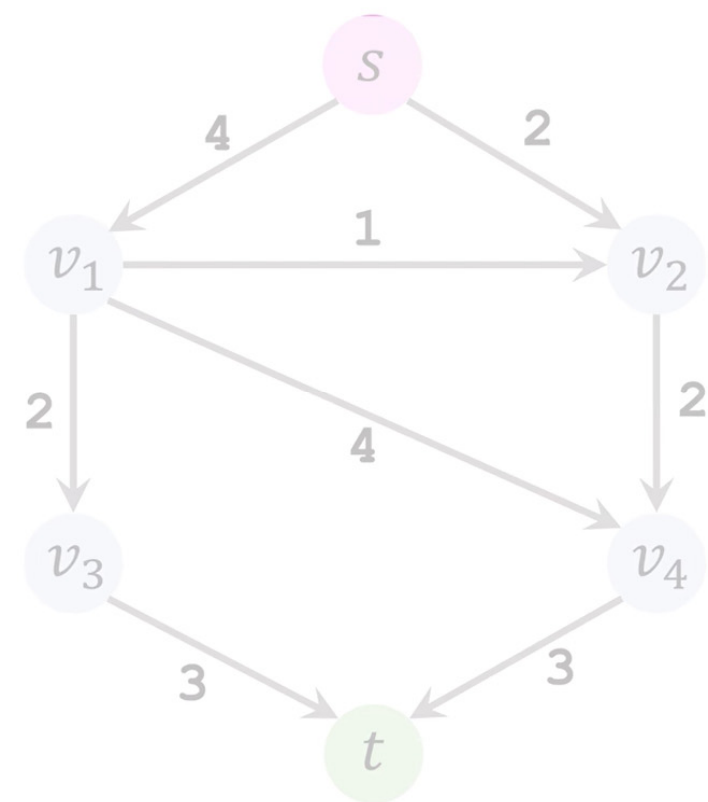
原始网络



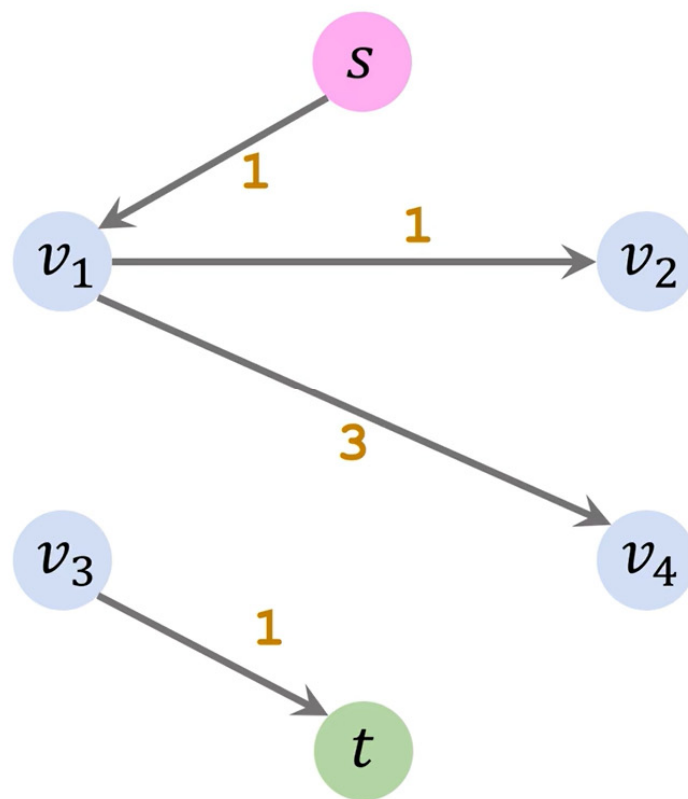
残流网络

网络流问题的求解算法

方法1: Naive Algorithm



原始网络

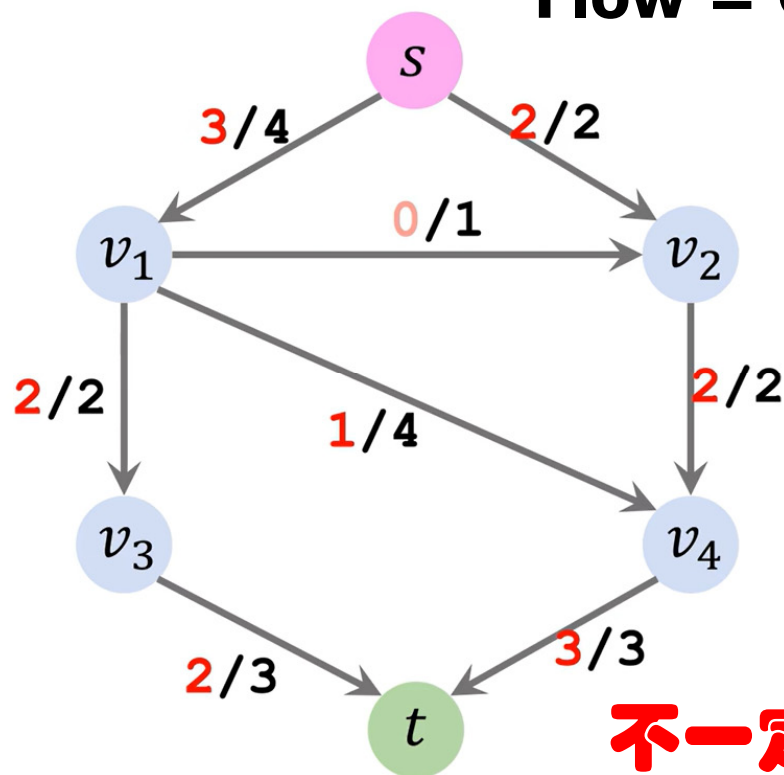


残流网络

网络流问题的求解算法

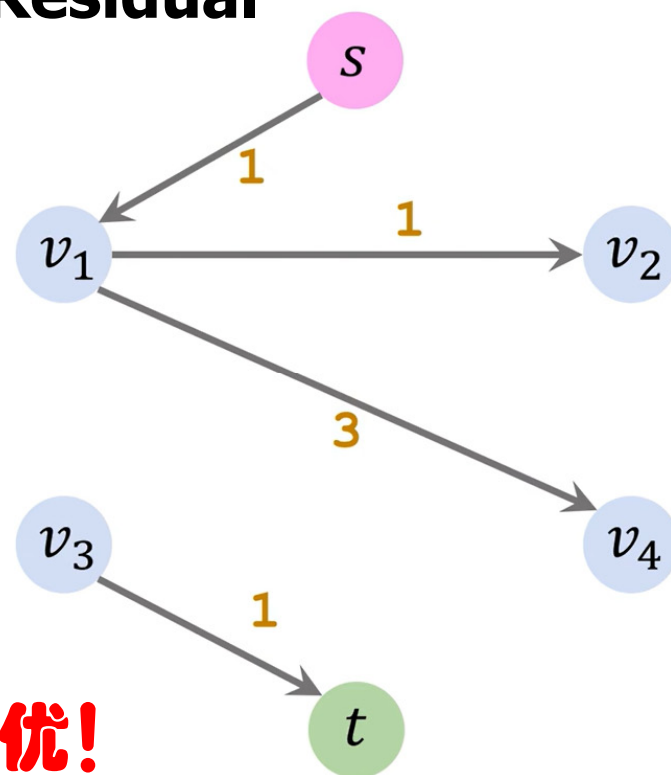
方法1: Naive Algorithm

$$\text{Flow} = \text{Capacity} - \text{Residual}$$



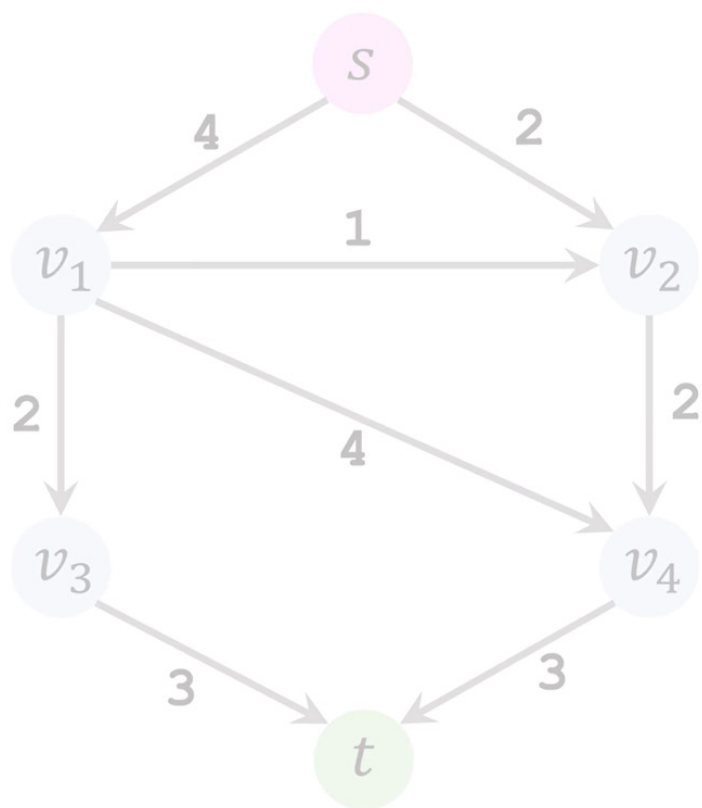
5

不一定能达到最优!

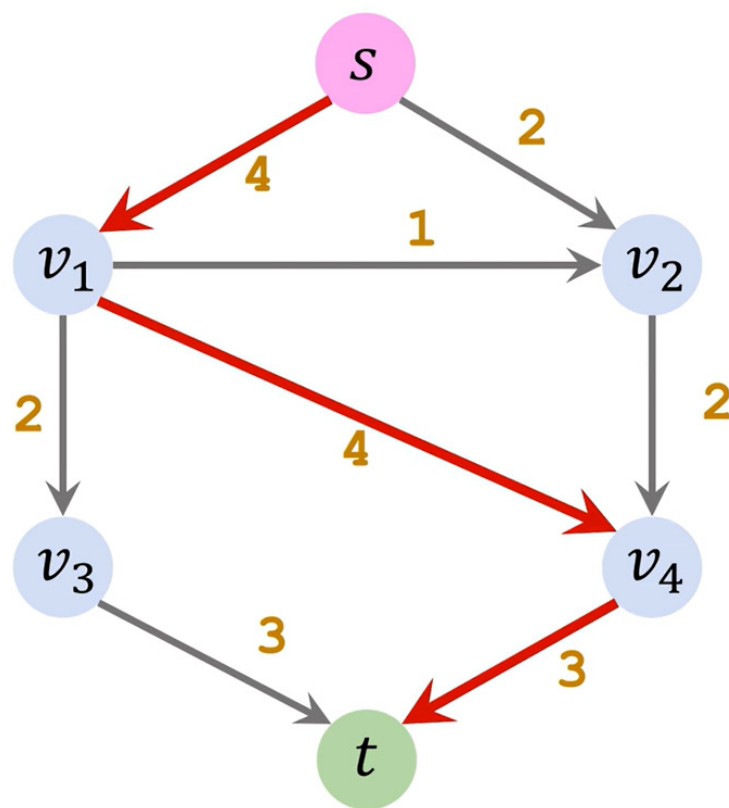


网络流问题的求解算法

方法1: Naive Algorithm



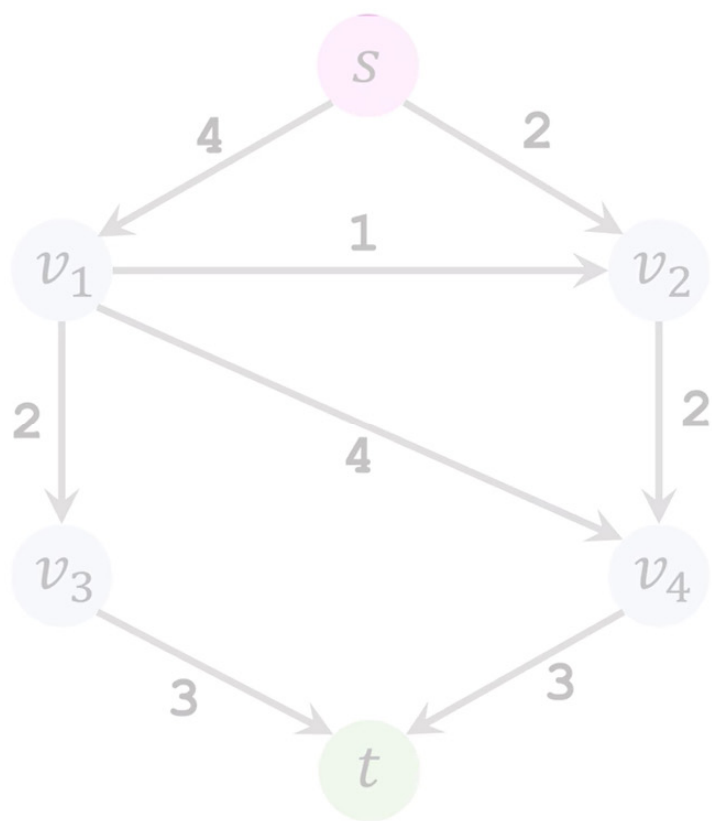
原始网络



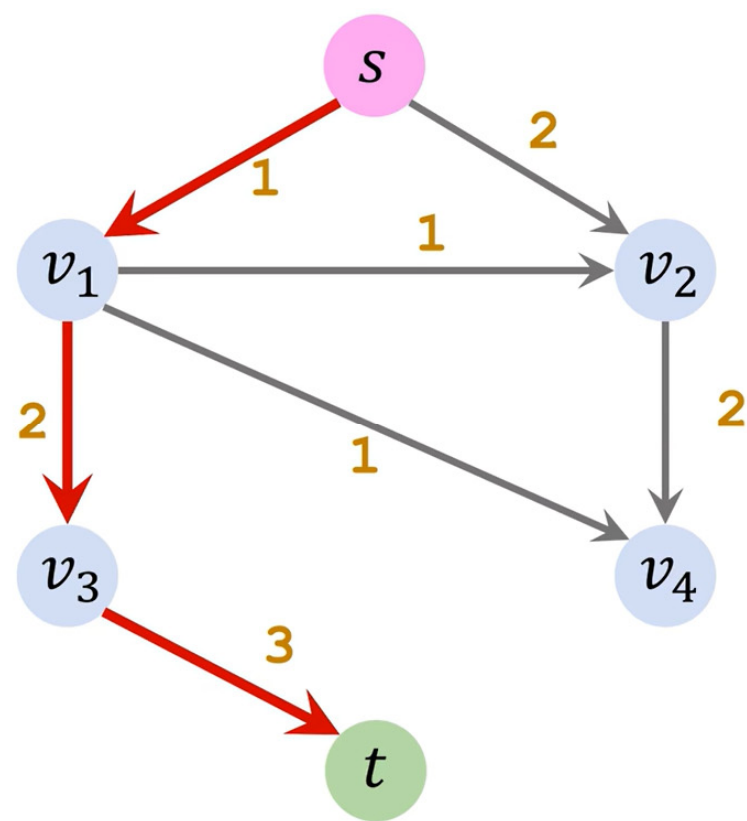
残流网络

网络流问题的求解算法

方法1: Naive Algorithm



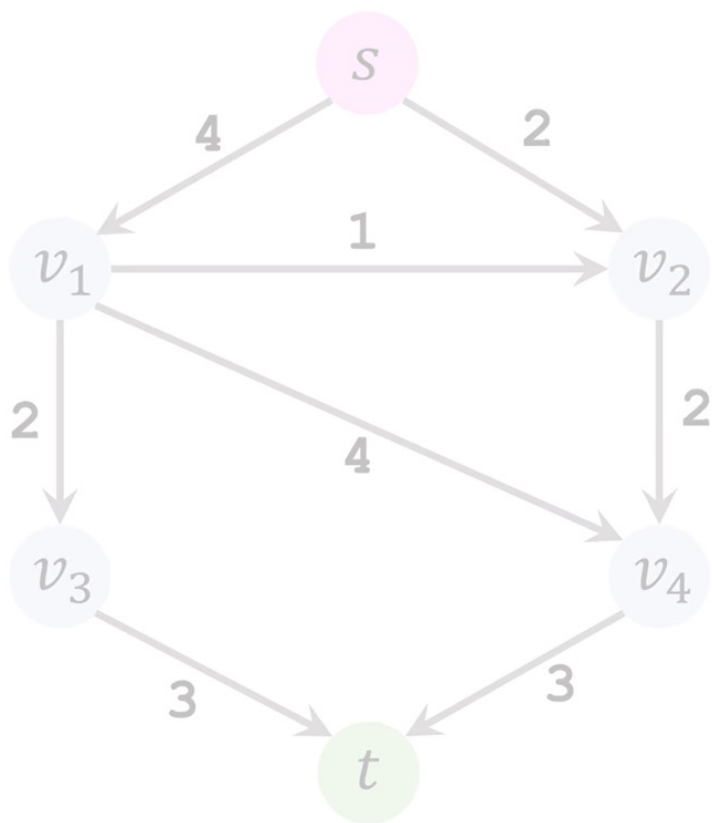
原始网络



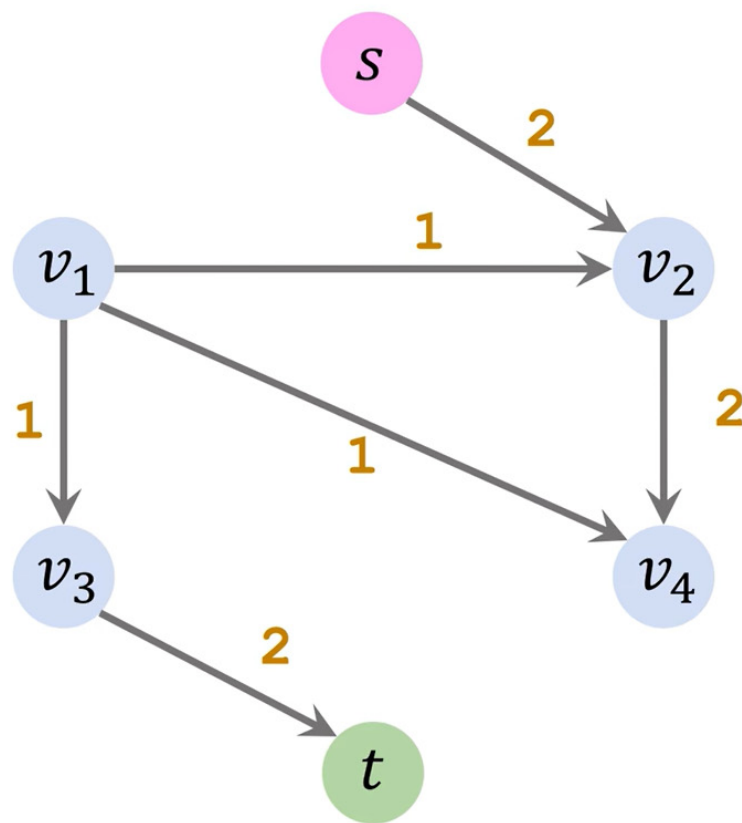
残流网络

网络流问题的求解算法

方法1: Naive Algorithm



原始网络

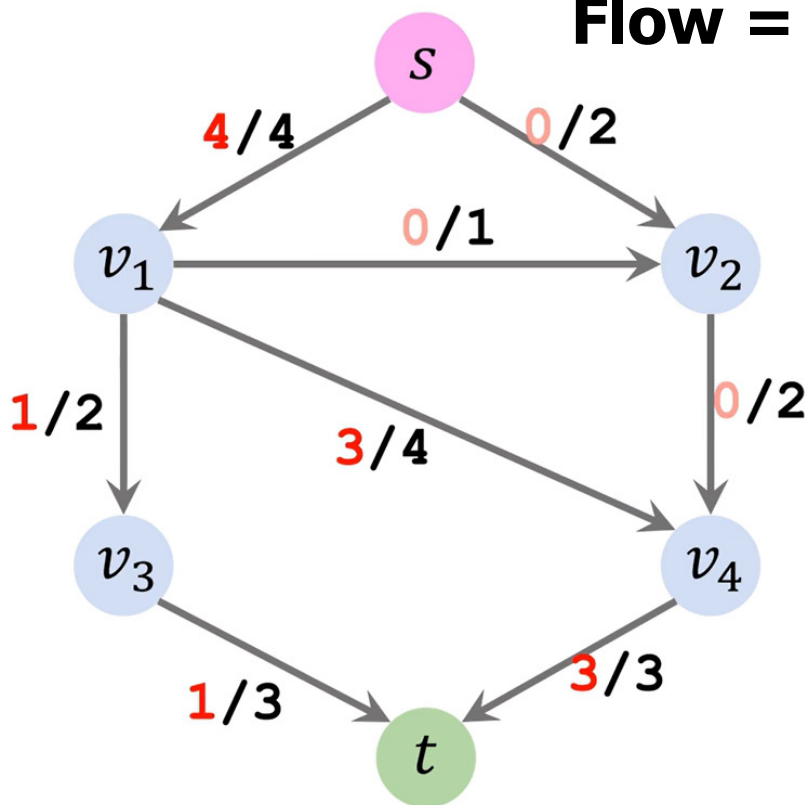


残流网络

网络流问题的求解算法

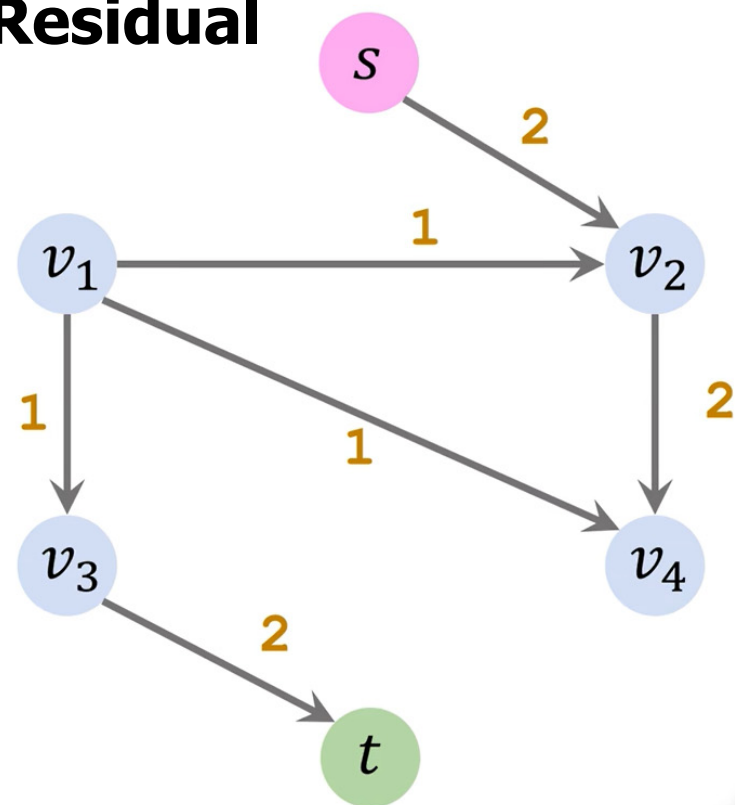
方法1: Naive Algorithm

$$\text{Flow} = \text{Capacity} - \text{Residual}$$



原始网络

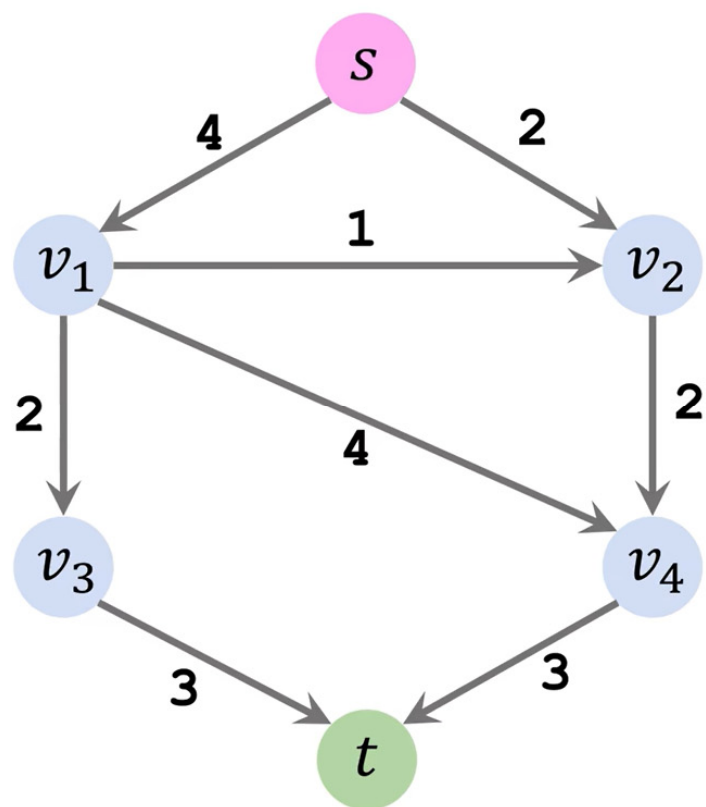
4



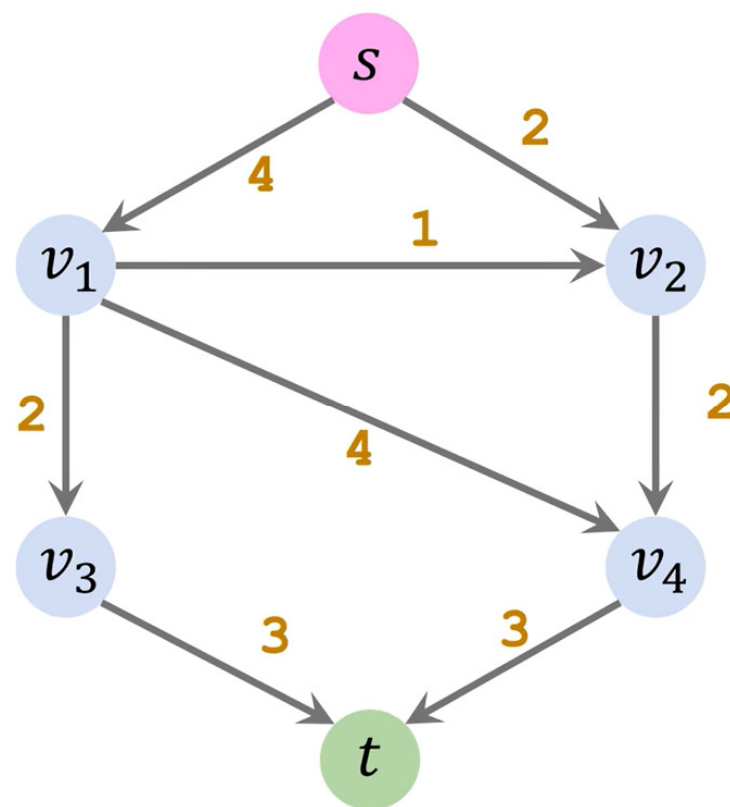
残流网络

网络流问题的求解算法

方法2: Ford-Fulkerson Algorithm



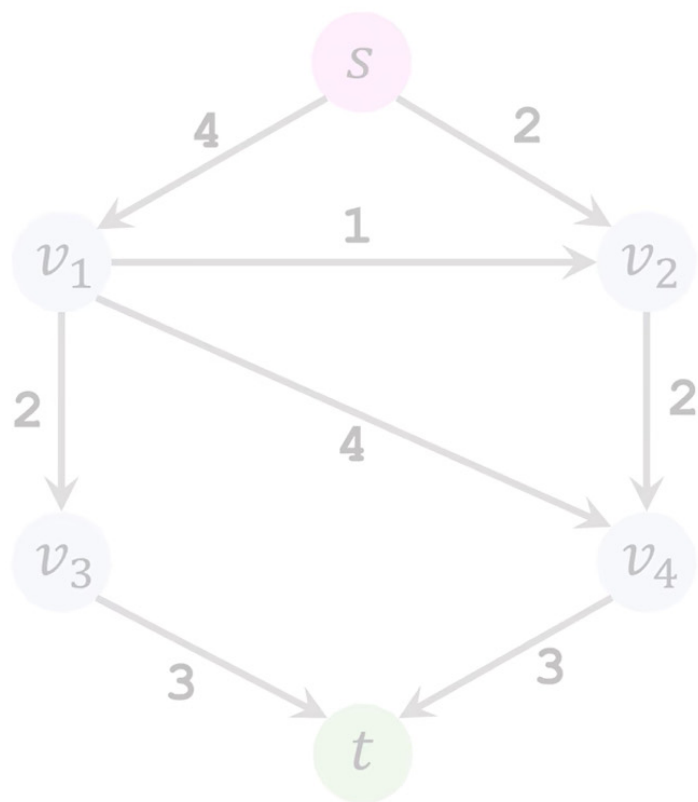
原始网络



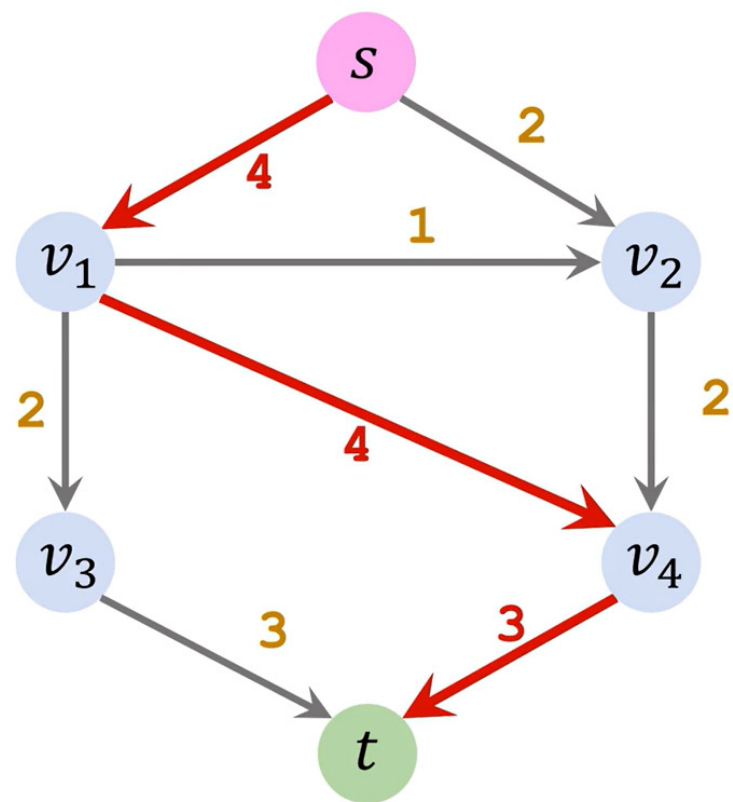
残流网络

网络流问题的求解算法

方法2: Ford-Fulkerson Algorithm



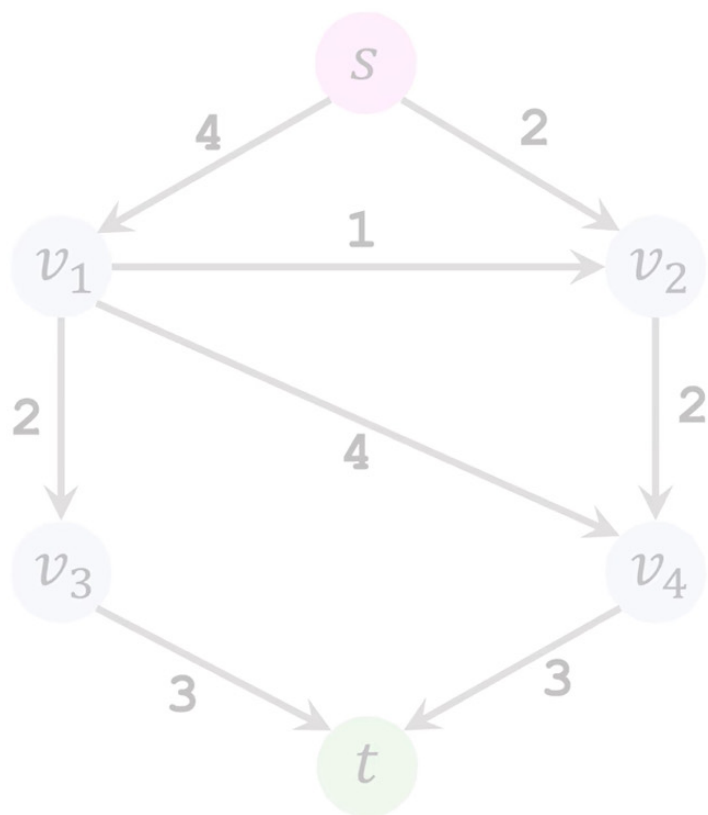
原始网络



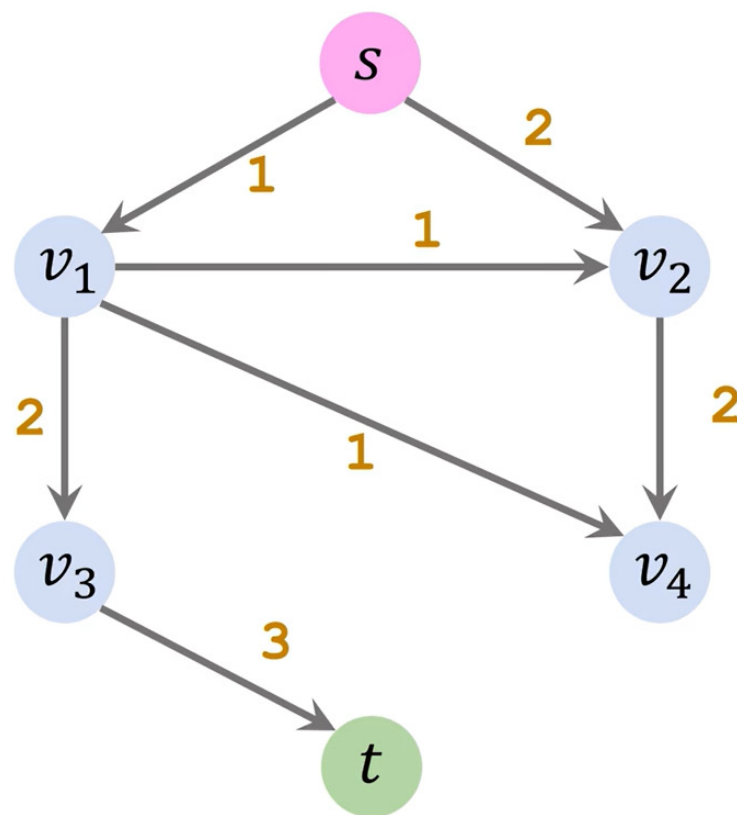
残流网络

网络流问题的求解算法

方法2: Ford-Fulkerson Algorithm



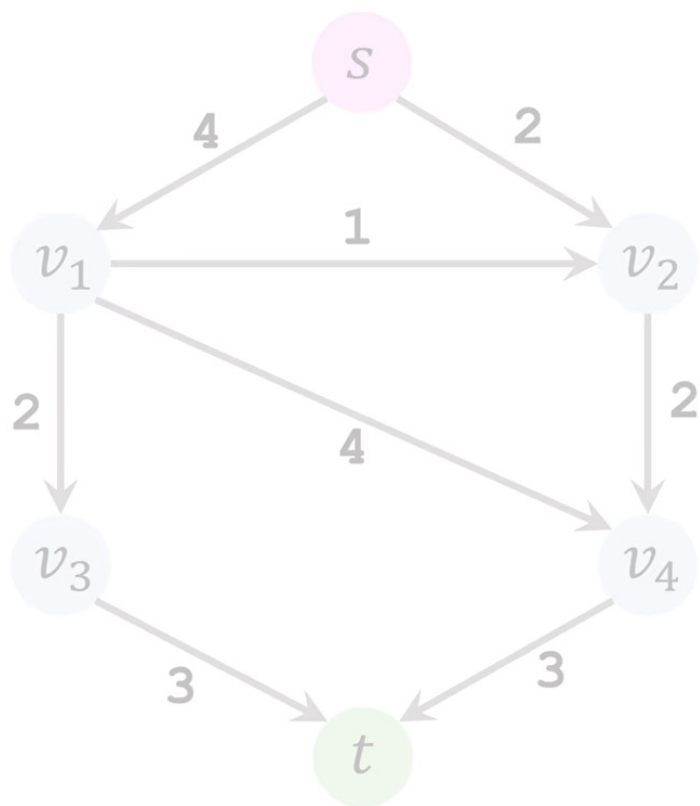
原始网络



残流网络

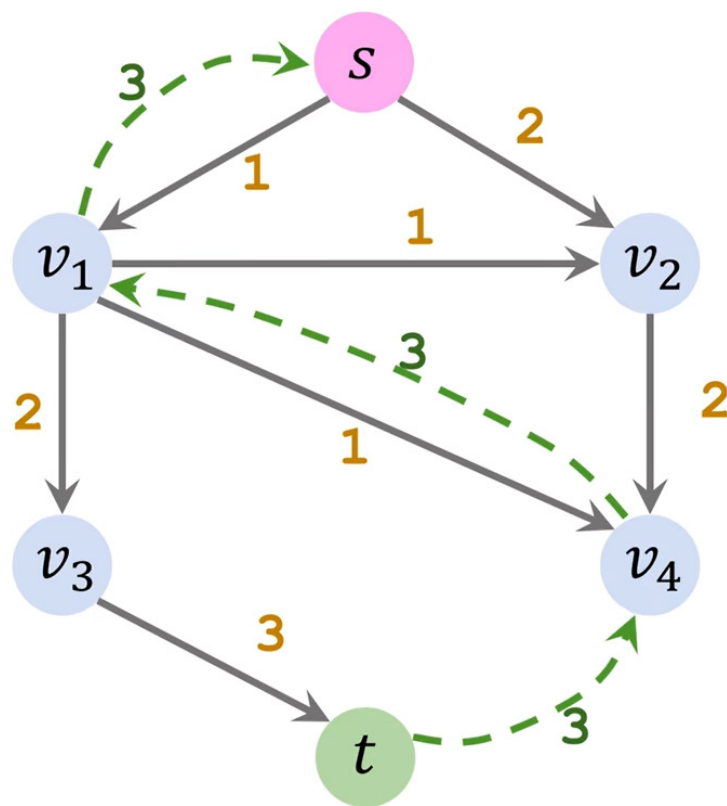
网络流问题的求解算法

方法2: Ford-Fulkerson Algorithm



原始网络

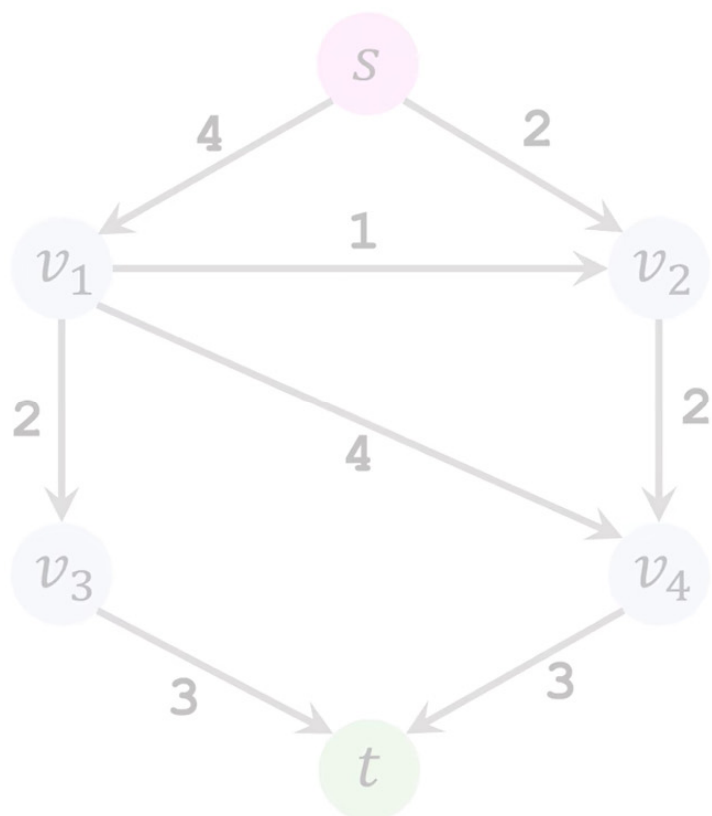
增广路径



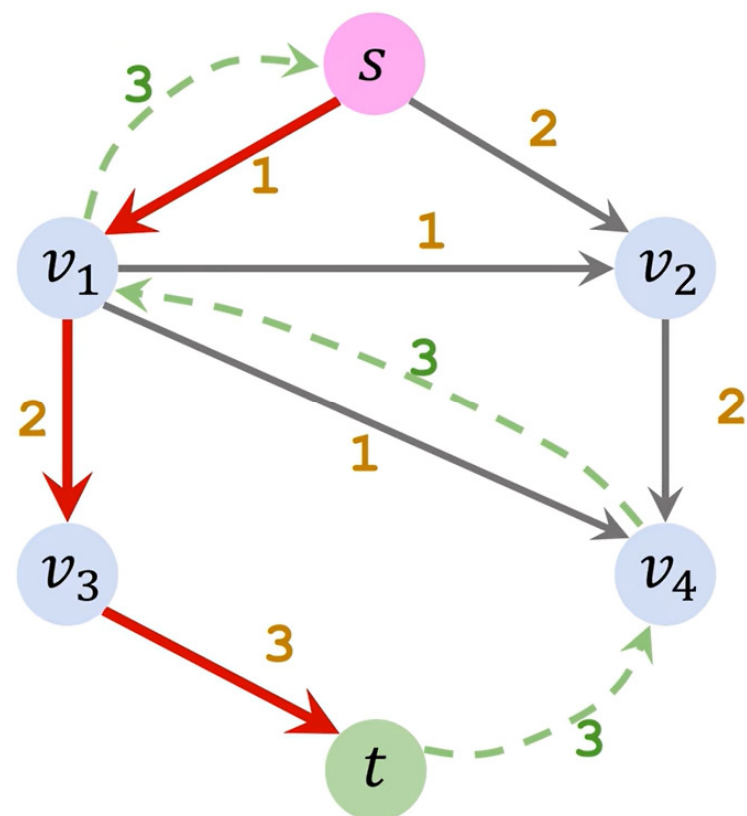
残流网络

网络流问题的求解算法

方法2: Ford-Fulkerson Algorithm



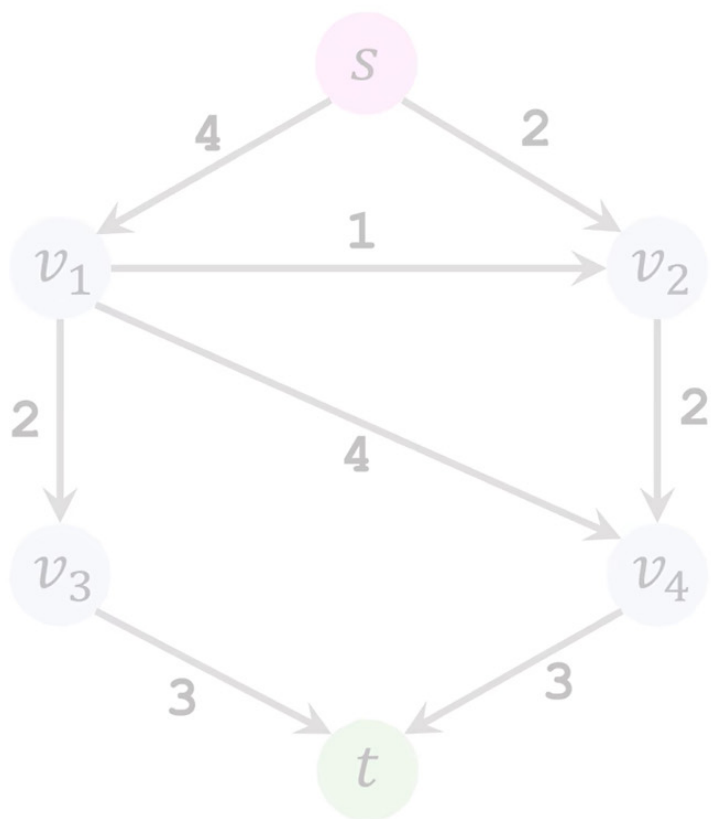
原始网络



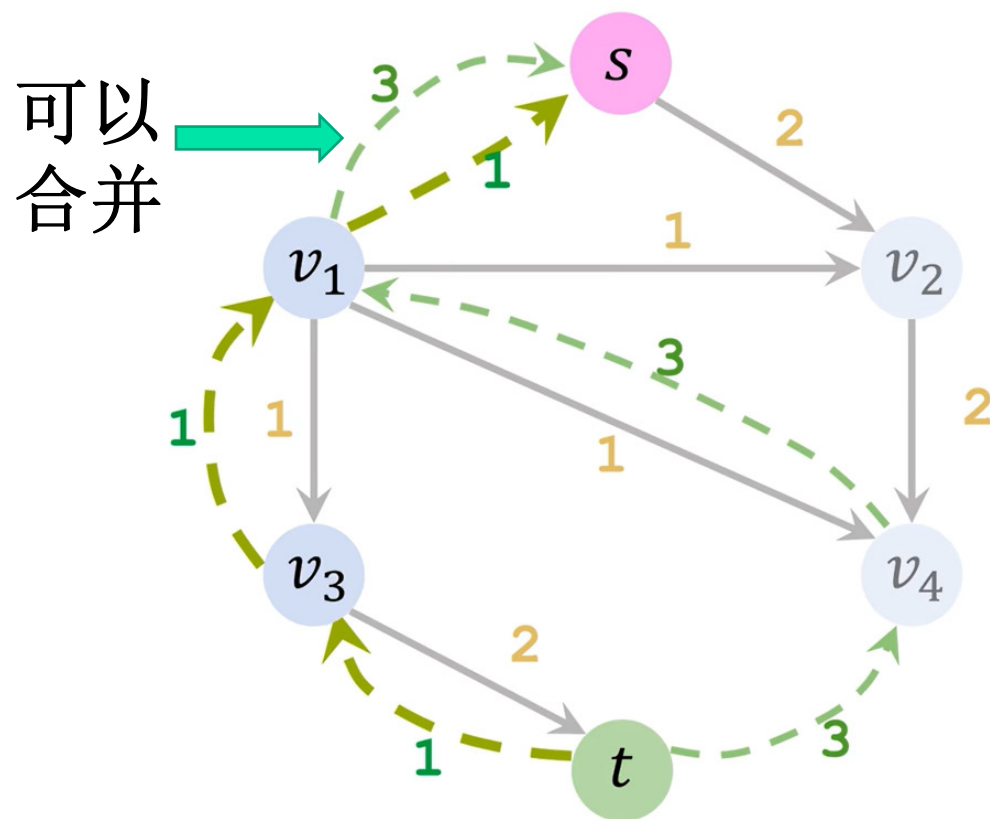
残流网络

网络流问题的求解算法

方法2: Ford-Fulkerson Algorithm



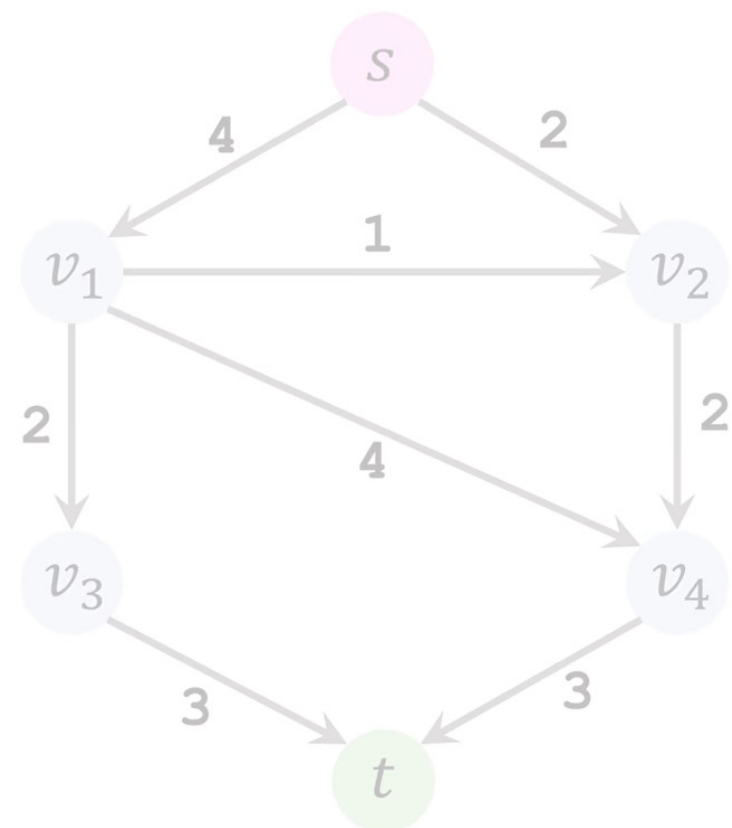
原始网络



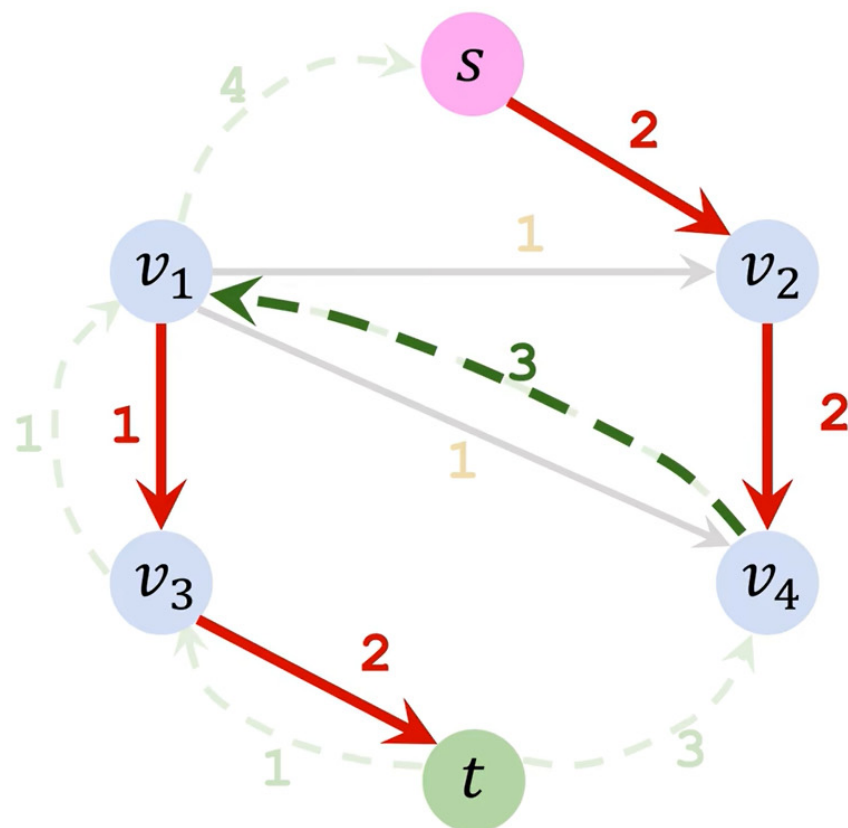
残流网络

网络流问题的求解算法

方法2: Ford-Fulkerson Algorithm



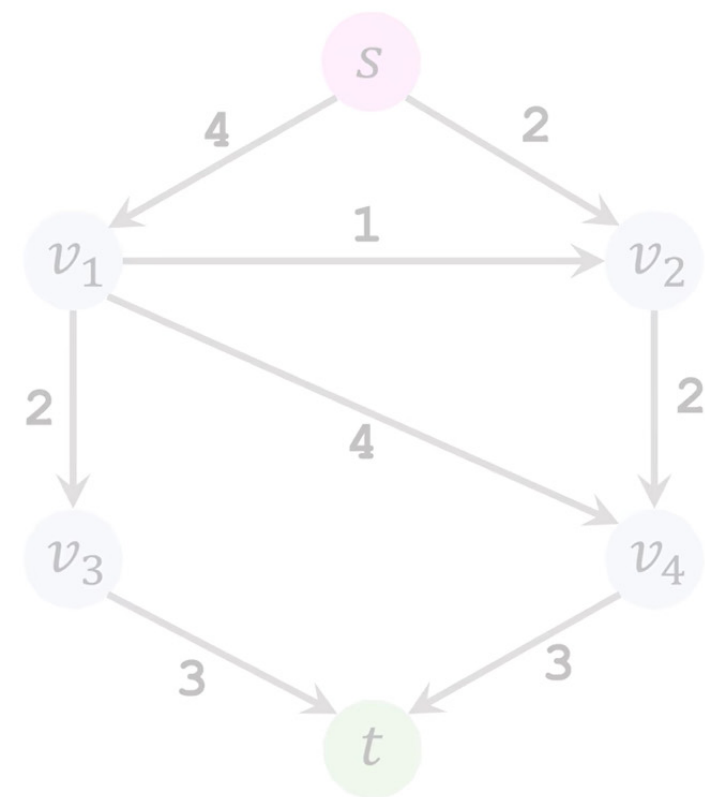
原始网络



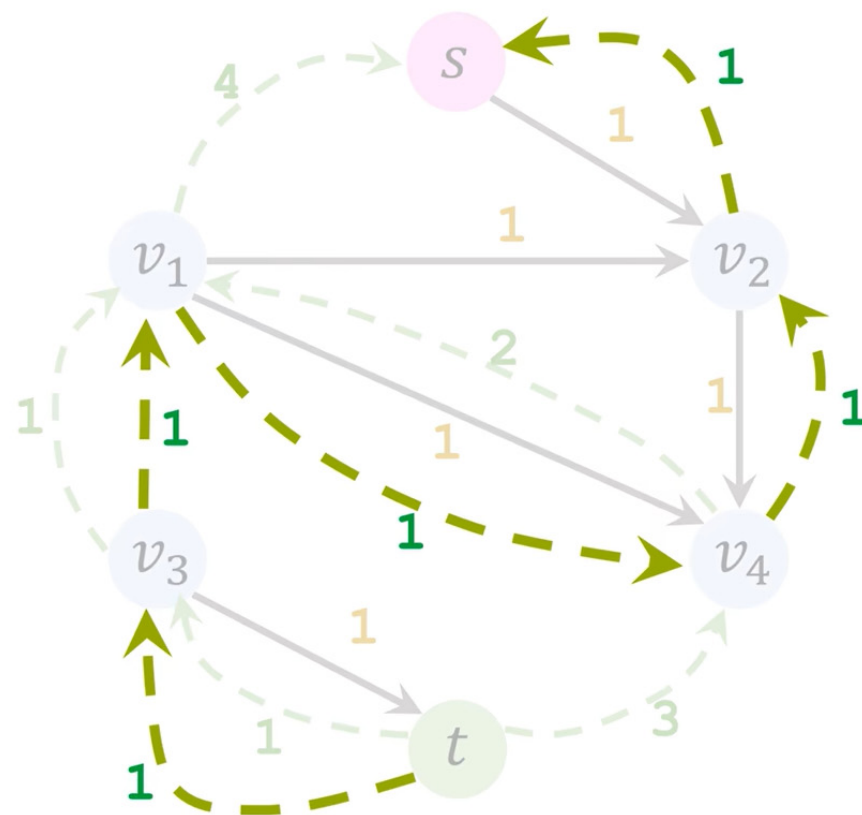
残流网络

网络流问题的求解算法

方法2: Ford-Fulkerson Algorithm



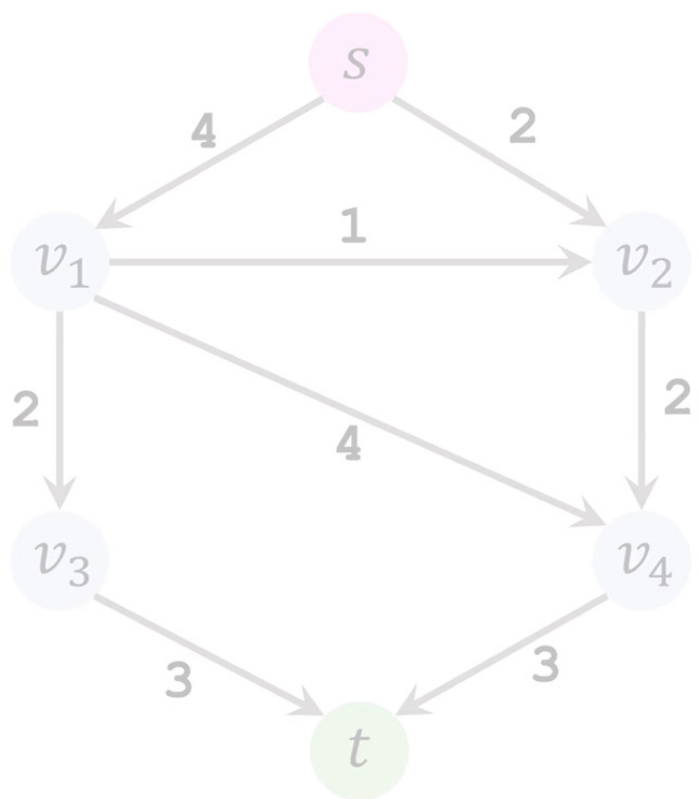
原始网络



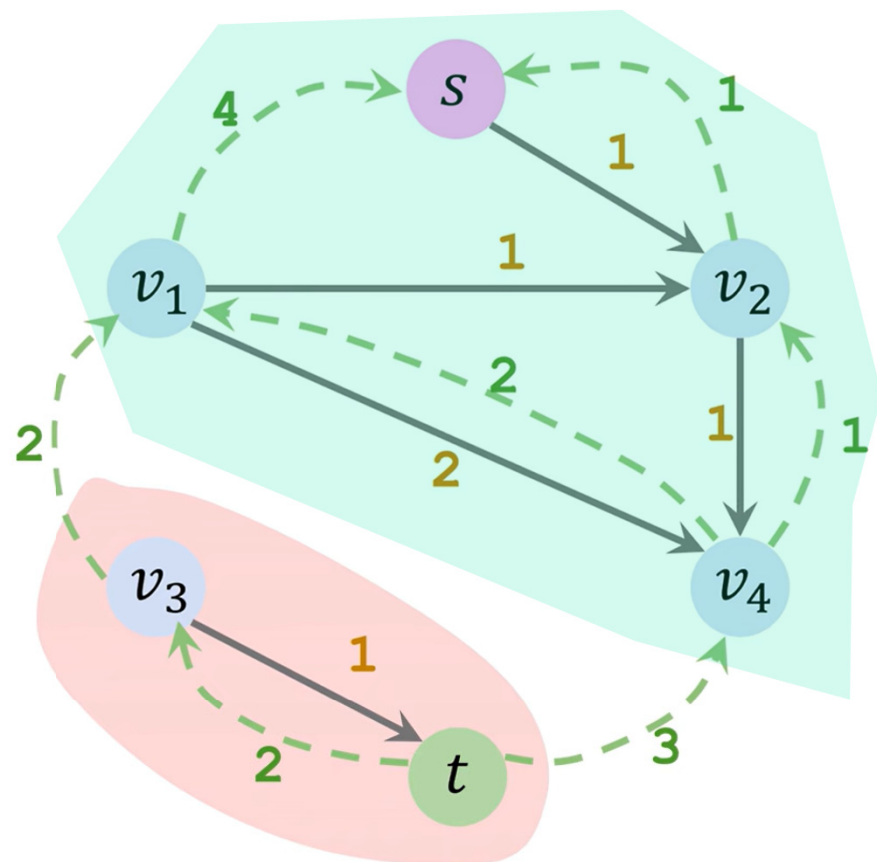
残流网络

网络流问题的求解算法

方法2: Ford-Fulkerson Algorithm



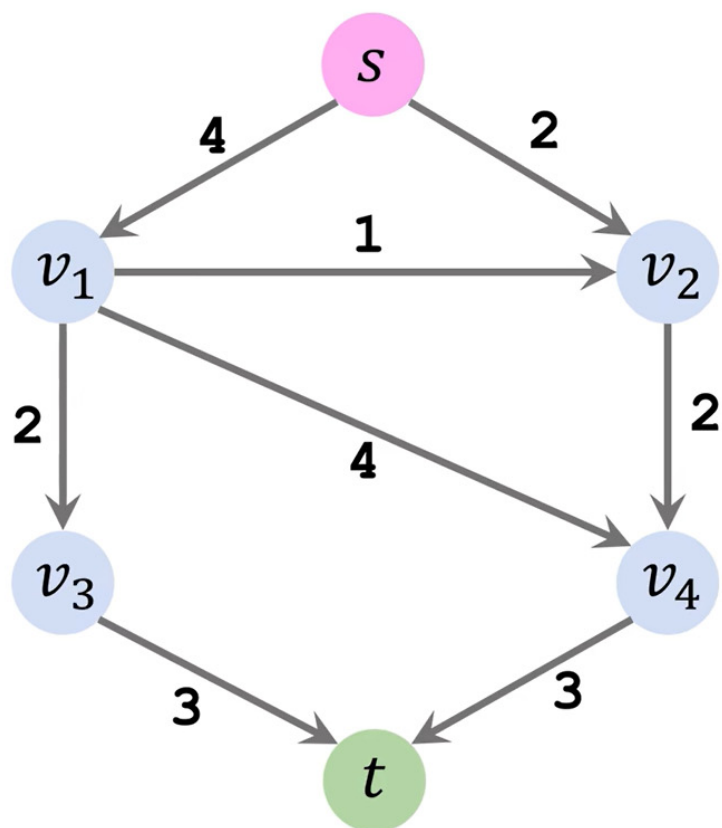
原始网络



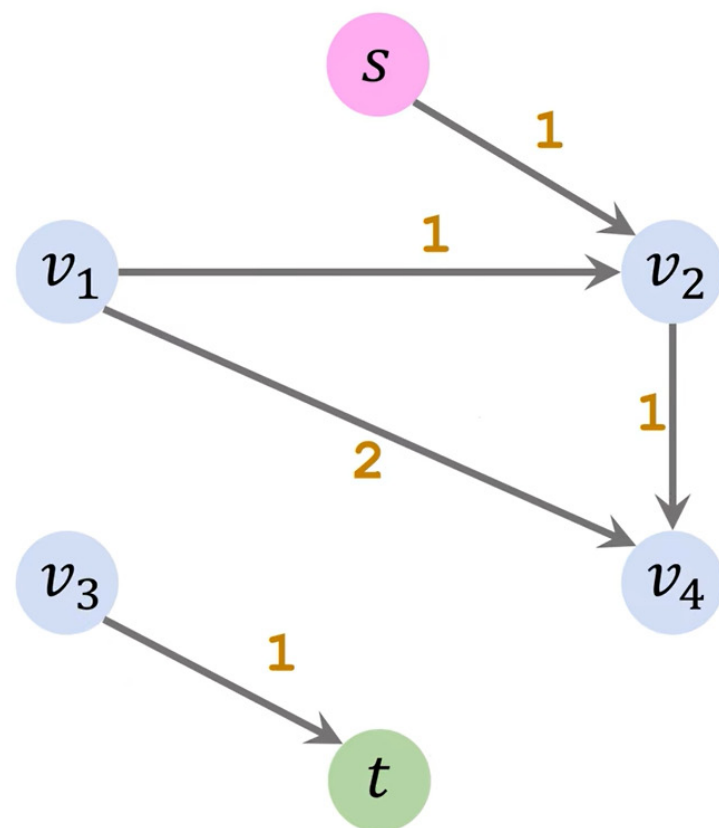
残流网络

网络流问题的求解算法

方法2: Ford-Fulkerson Algorithm



原始网络

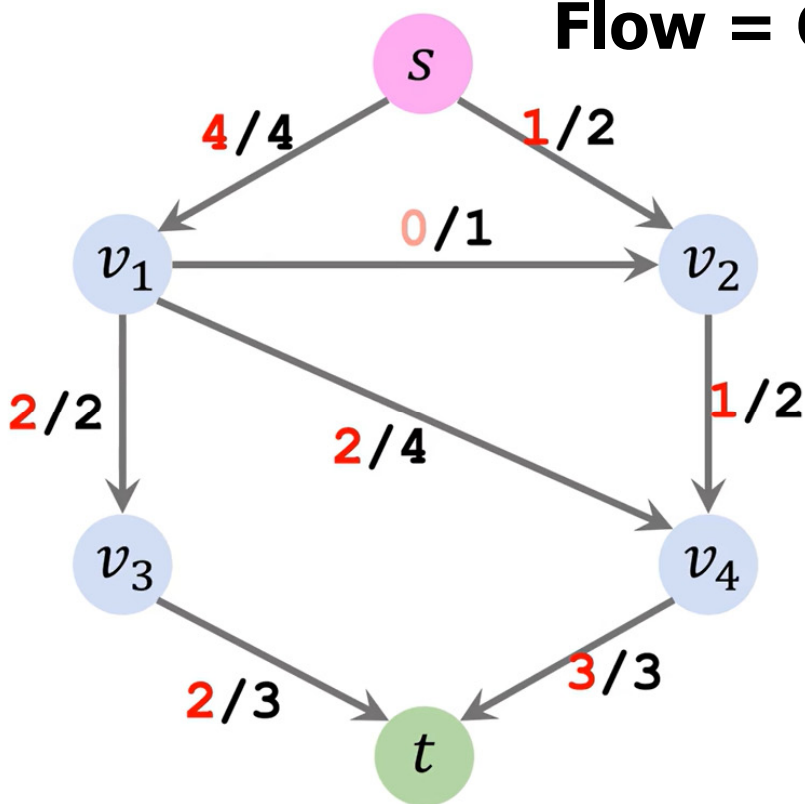


残流网络

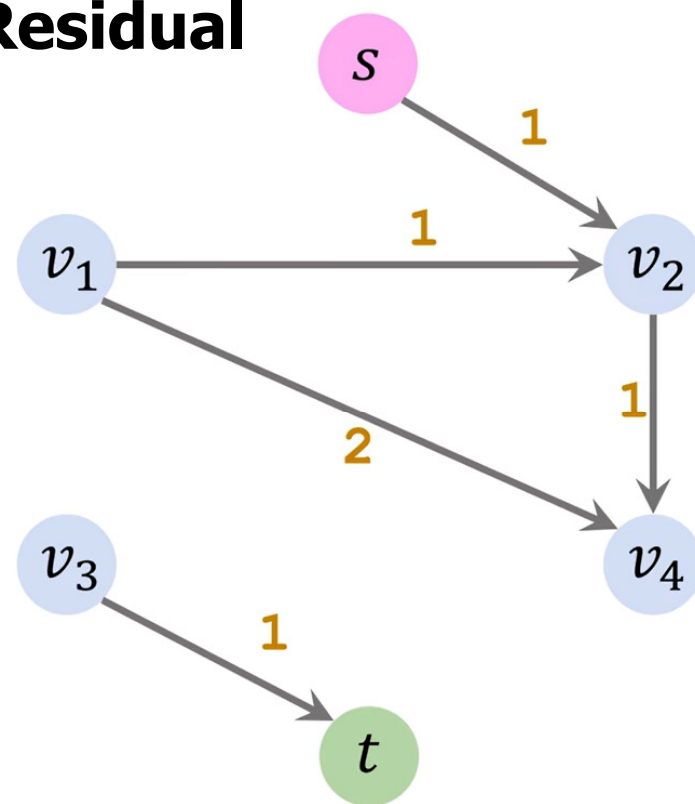
网络流问题的求解算法

方法2: Ford-Fulkerson Algorithm

Flow = Capacity - Residual



5



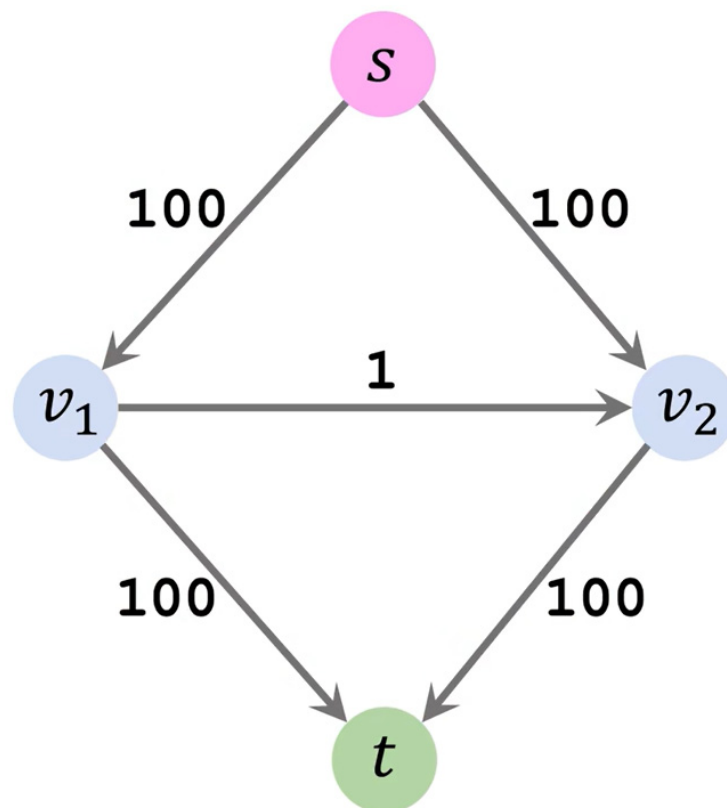
网络流问题的求解算法

方法2: Ford-Fulkerson Algorithm

时间复杂度?

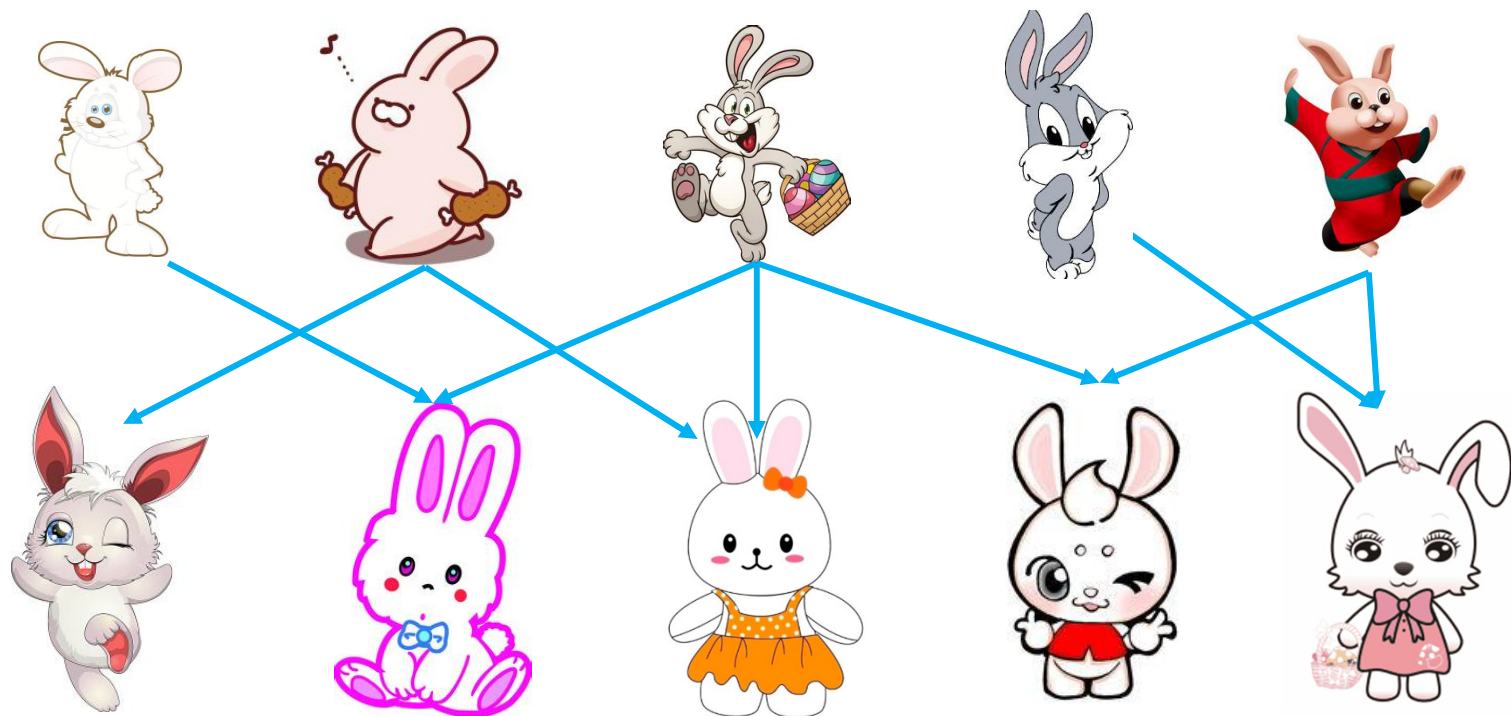
假设有 m 条边, 最大流是 f

$O(fm)$



回顾第一次课的兔子匹配问题

草原上生活着一群可爱的兔子，有5只公兔子和5只母兔子，每只公兔子都对若干母兔子有好感。如何帮助尽可能多的公兔子找到伴侣？



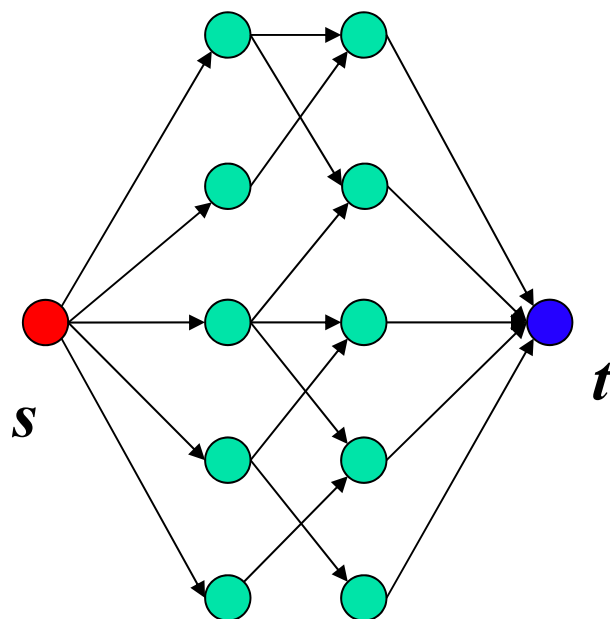
兔子匹配—建模为最大流问题

■ 最大流的网络

- 将公兔子和母兔子看作图中的点
- 若公兔子 a 喜欢母兔子 b ，则在 a 和 b 之间连一条边，边的容量为1
- 增加一个源点 s 和汇点 t

■ 问题描述

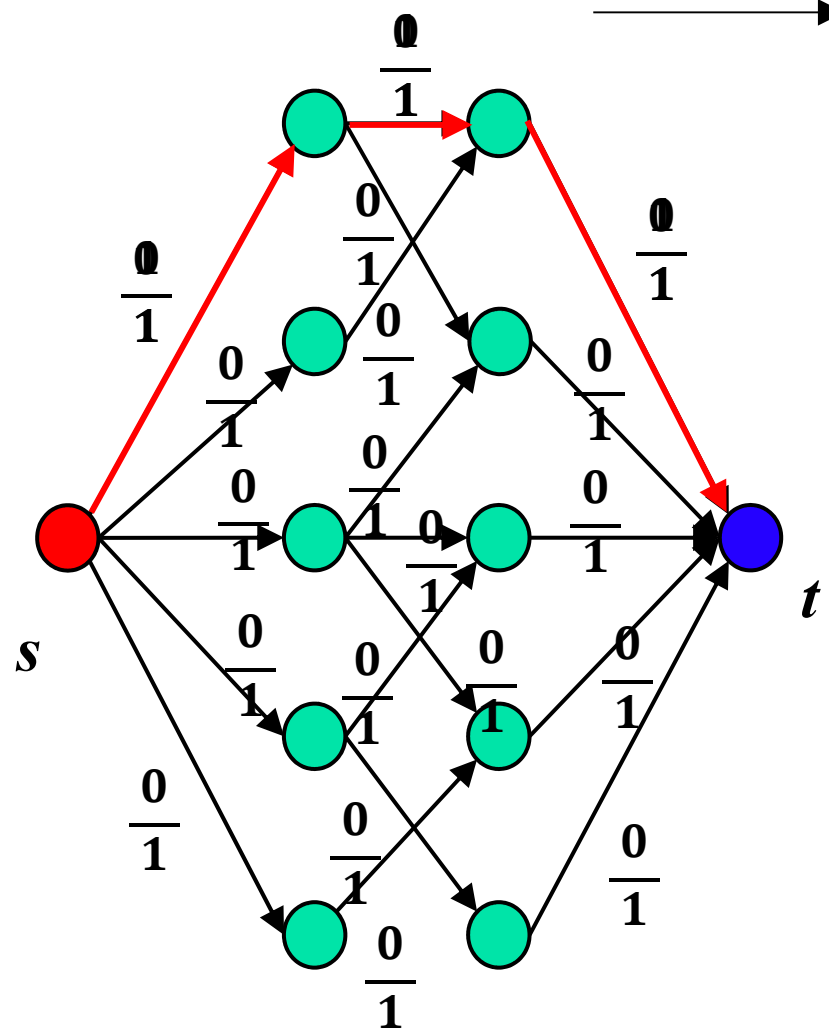
- 找从源点 s 到汇点 t 的最大流
- 最大流流量就是最大配对数



兔子匹配—最大流算法

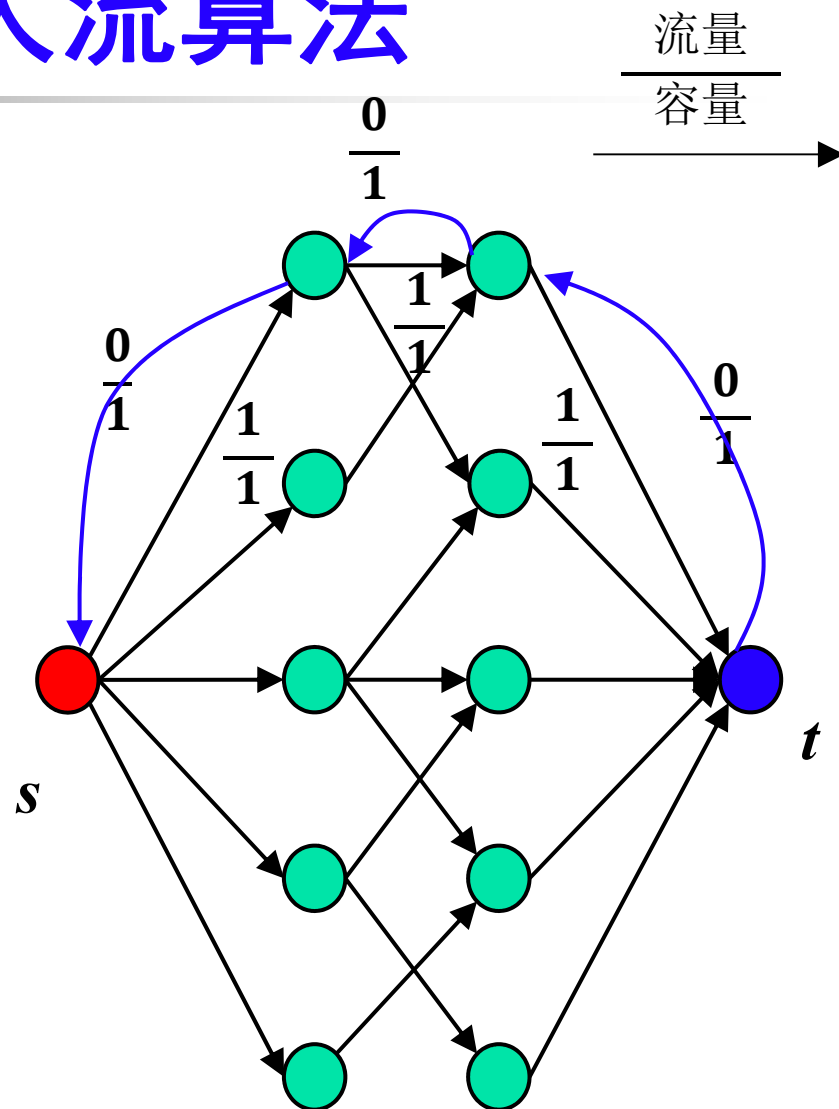
流量
容量

- 1) 找一条从s到t的流



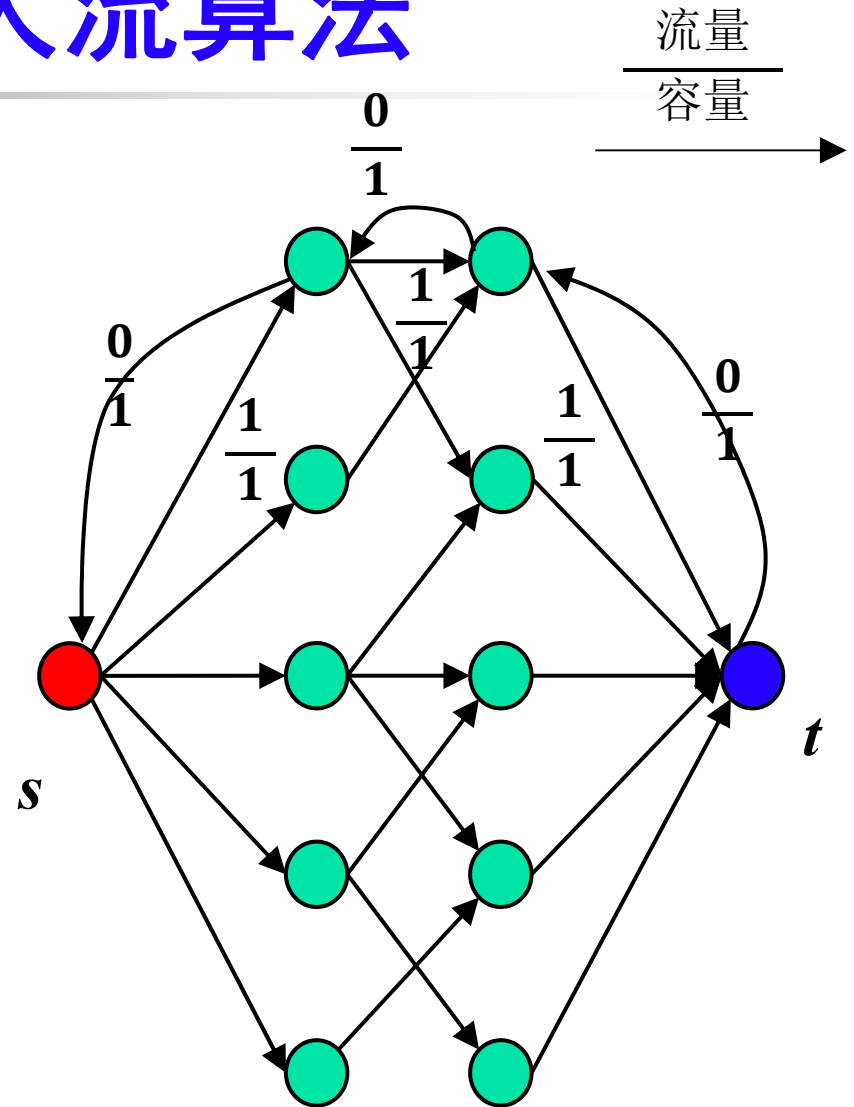
兔子匹配—最大流算法

- 1) 找一条从s到t的流
- 2) 构建剩余网络
 - 若边 e 有流量 $\text{flow}(e)$
 - 则生成 e 的反向边 e'
 - 边 e' 的容量为 $\text{cap}(e')$, 流量为0
 - 剩余网络的作用是让流可以被回退



兔子匹配—最大流算法

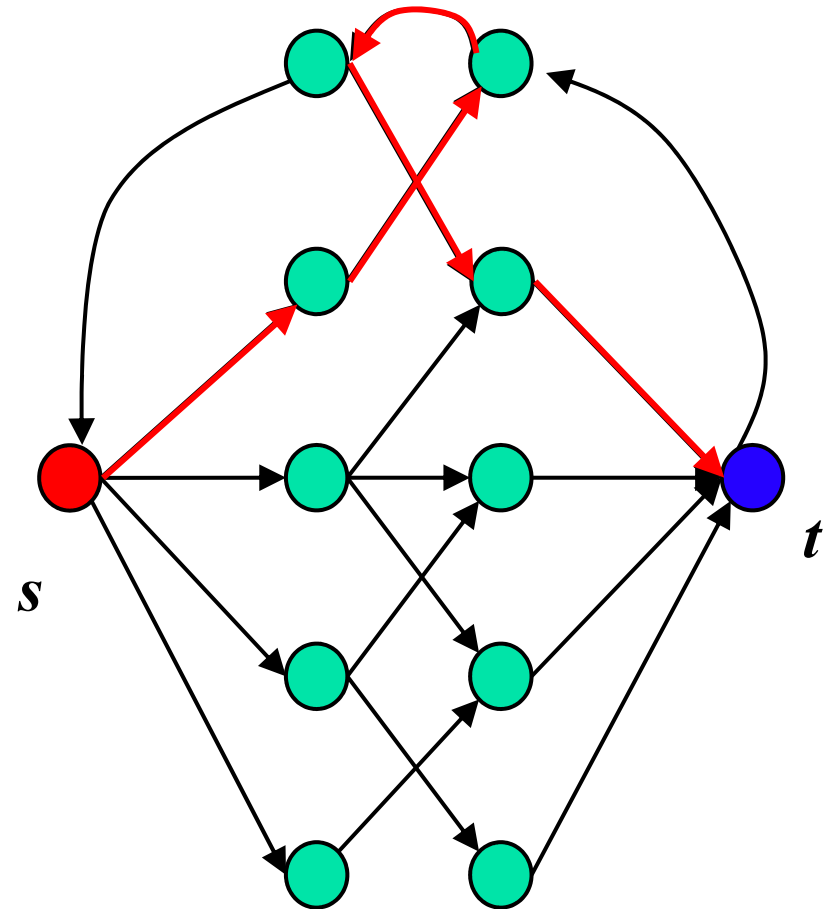
- 1) 找一条从s到t的流
- 2) 构建剩余网络
- 3) 在剩余网络中重复前两步，直到再也找不到从s到t的流



兔子匹配—最大流算法

- 1) 找一条从s到t的流
- 2) 构建剩余网络
- 3) 在剩余网络中重复前两步，直到再也找不到从s到t的流

下图仅显示
有空余容量的边

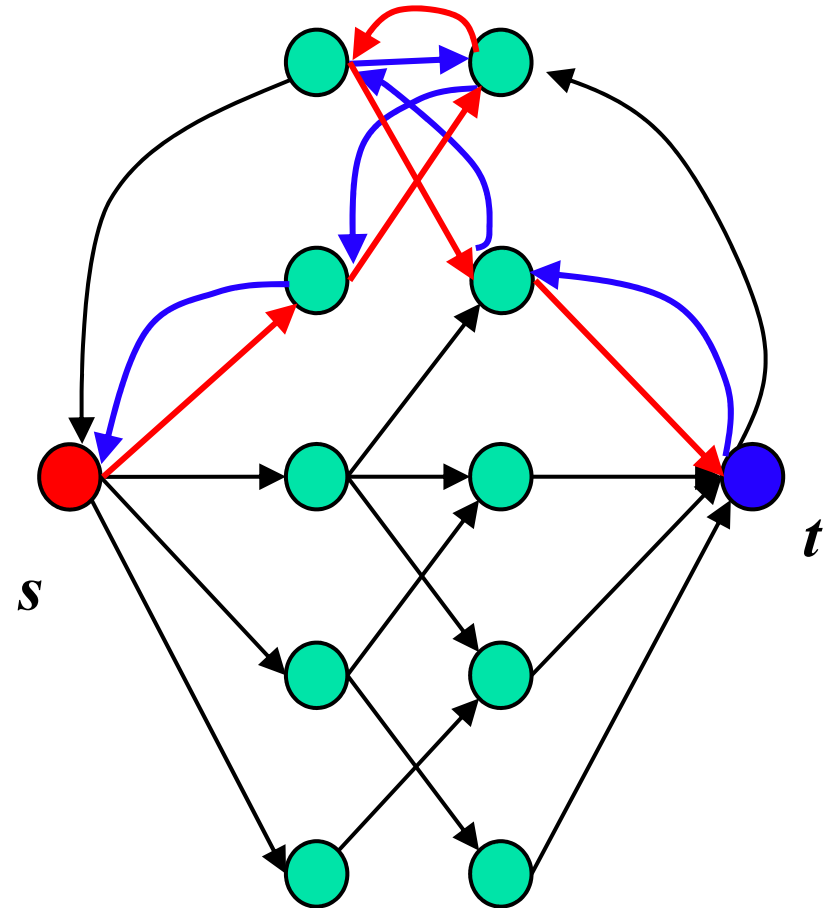


找一条从s到t的流

兔子匹配—最大流算法

- 1) 找一条从s到t的流
- 2) 构建剩余网络
- 3) 在剩余网络中重复前两步，直到再也找不到从s到t的流

下图仅显示
有空余容量的边

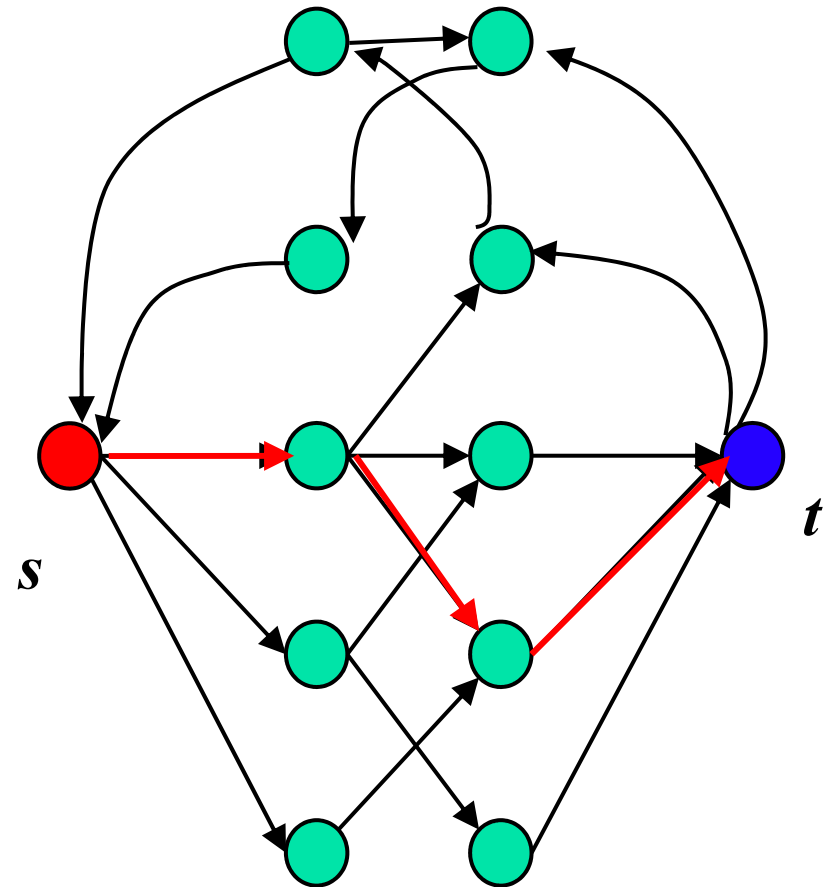


构建剩余网络

兔子匹配—最大流算法

- 1) 找一条从s到t的流
- 2) 构建剩余网络
- 3) 在剩余网络中重复前两步，直到再也找不到从s到t的流

下图仅显示
有空余容量的边

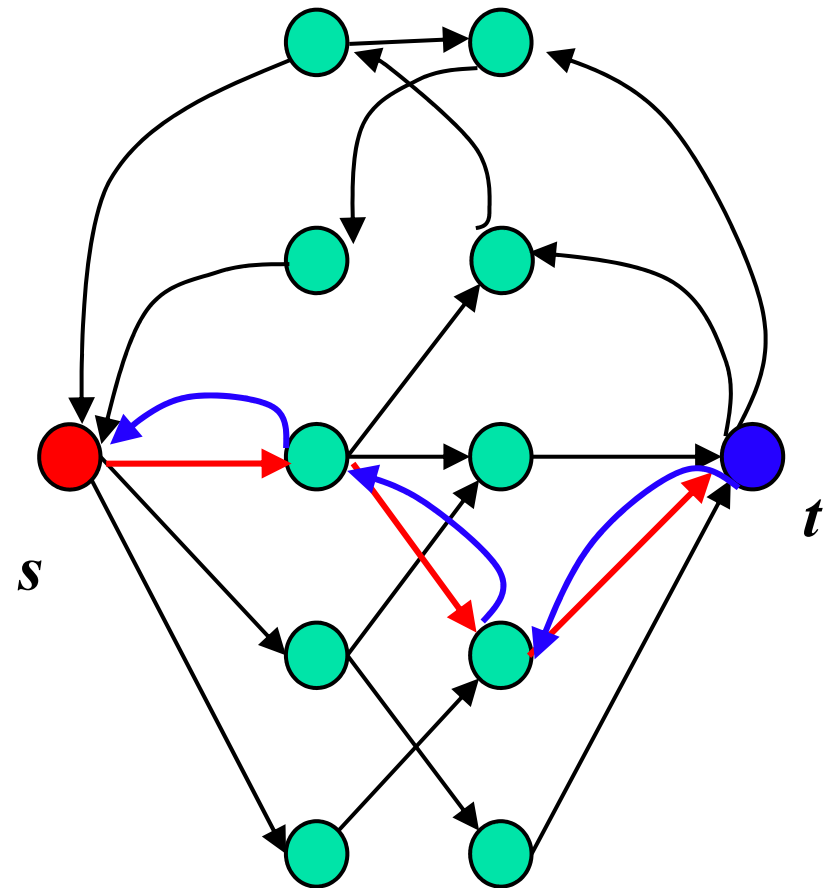


找一条从s到t的流

兔子匹配—最大流算法

- 1) 找一条从s到t的流
- 2) 构建剩余网络
- 3) 在剩余网络中重复前两步，直到再也找不到从s到t的流

下图仅显示
有空余容量的边

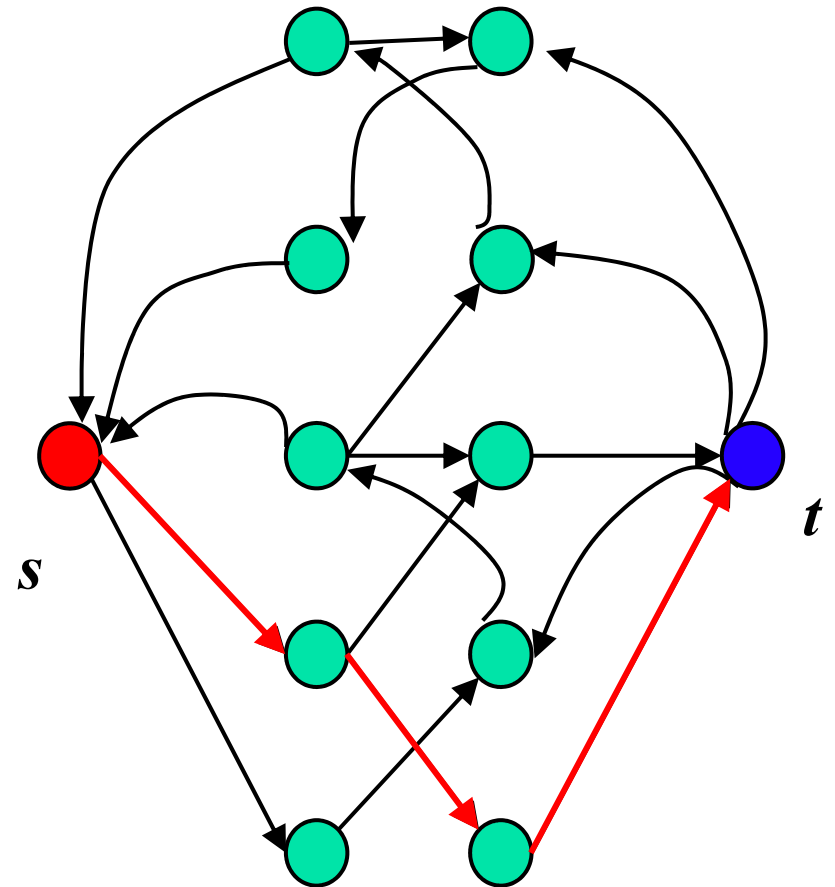


构建剩余网络

兔子匹配—最大流算法

- 1) 找一条从s到t的流
- 2) 构建剩余网络
- 3) 在剩余网络中重复前两步，直到再也找不到从s到t的流

下图仅显示
有空余容量的边

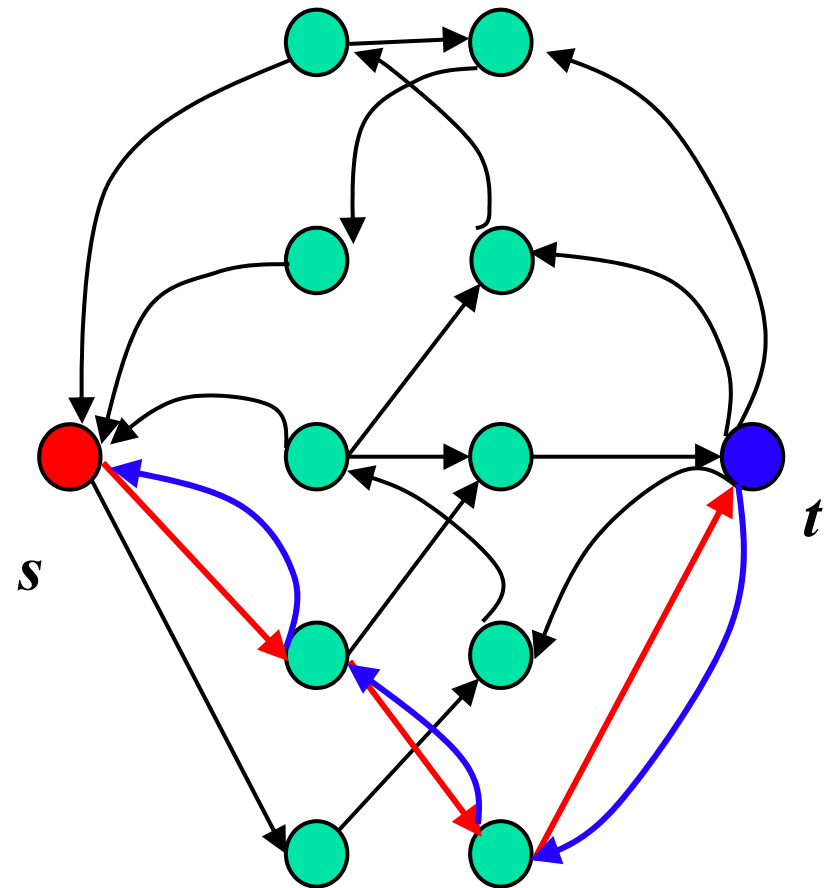


找一条从s到t的流

兔子匹配—最大流算法

- 1) 找一条从s到t的流
- 2) 构建剩余网络
- 3) 在剩余网络中重复前两步，直到再也找不到从s到t的流

下图仅显示
有空余容量的边

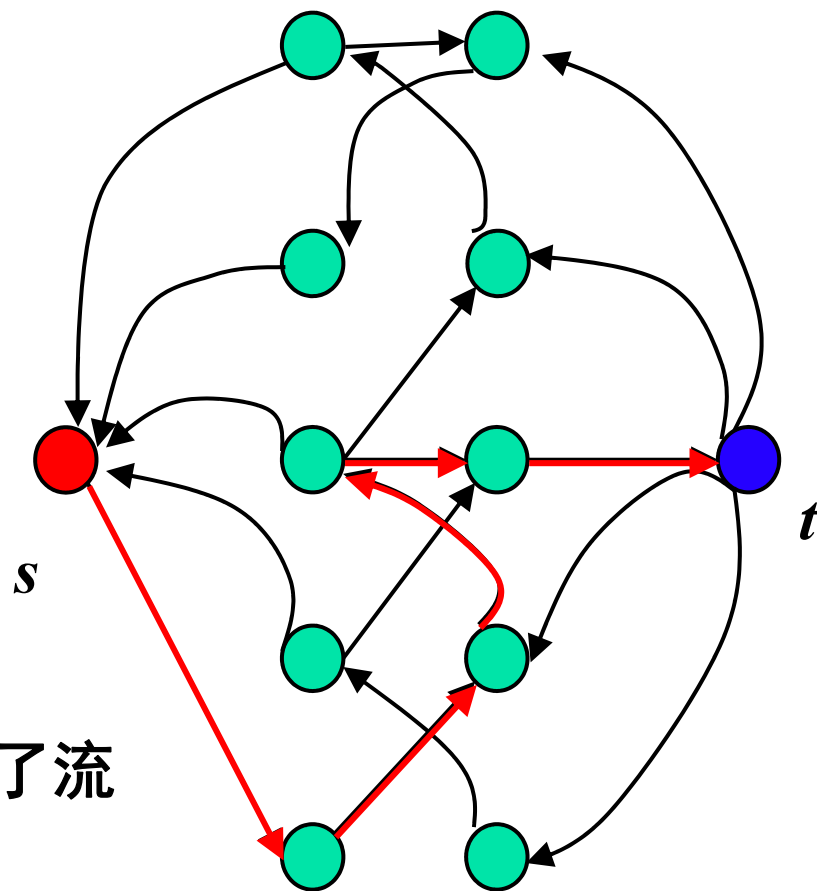


构建剩余网络

兔子匹配—最大流算法

- 1) 找一条从s到t的流
- 2) 构建剩余网络
- 3) 在剩余网络中重复前两步，直到再也找不到从s到t的流

下图仅显示
有空余容量的边



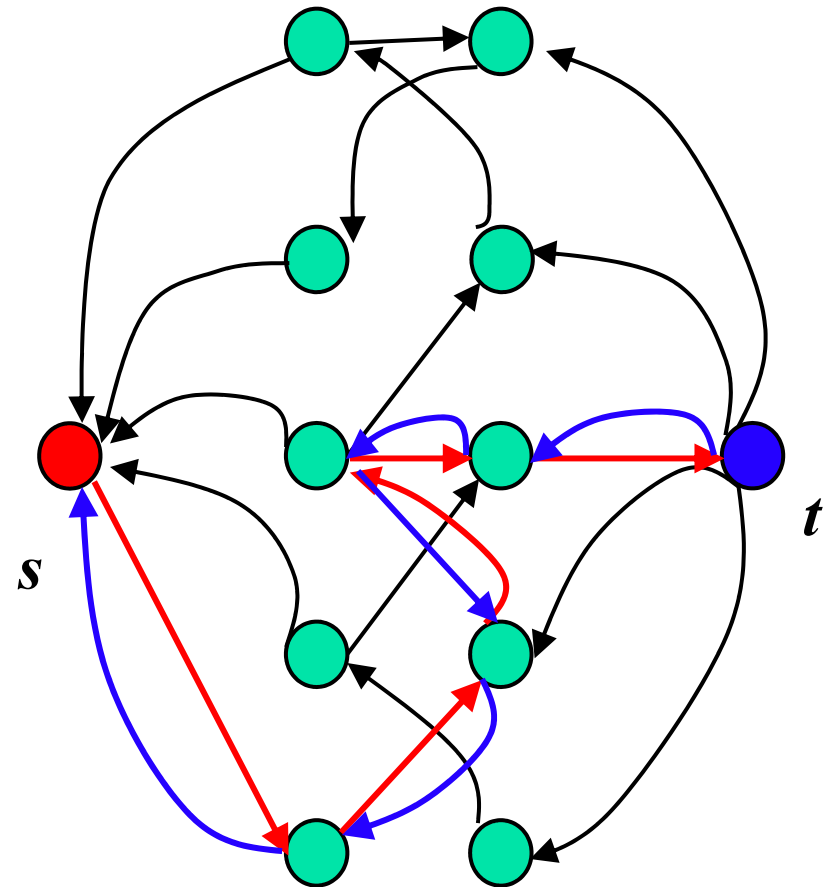
注意： 此处是在剩余网络中回退了流

找一条从s到t的流

兔子匹配—最大流算法

- 1) 找一条从s到t的流
- 2) 构建剩余网络
- 3) 在剩余网络中重复前两步，直到再也找不到从s到t的流

下图仅显示
有空余容量的边

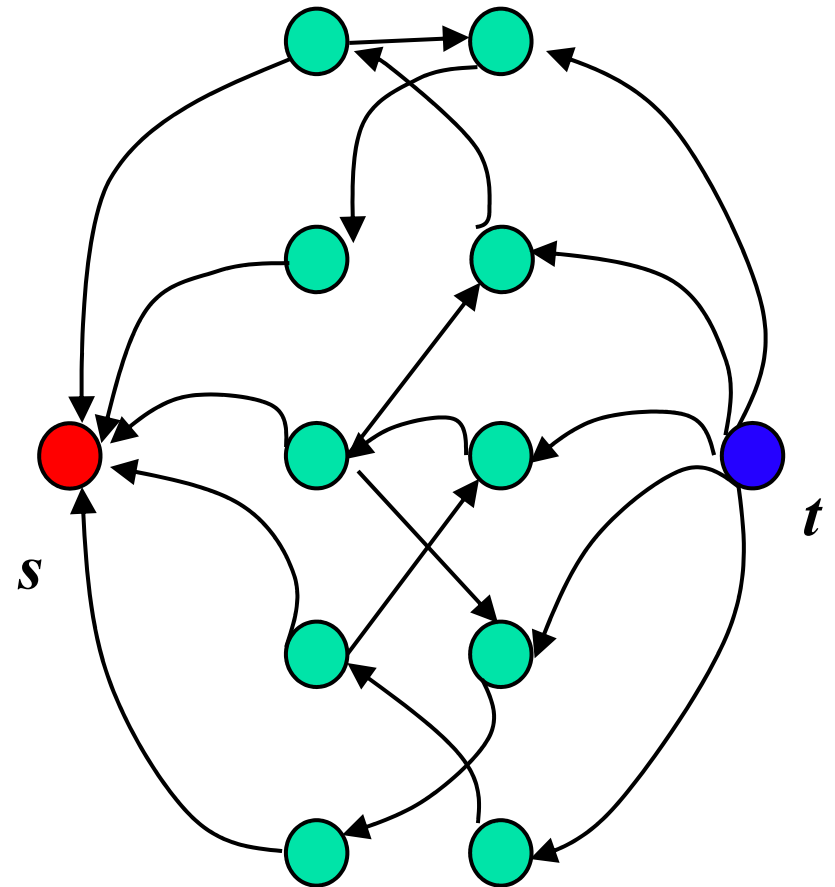


找一条从s到t的流

兔子匹配—最大流算法

- 1) 找一条从s到t的流
- 2) 构建剩余网络
- 3) 在剩余网络中重复前两步，直到再也找不到从s到t的流

下图仅显示
有空余容量的边

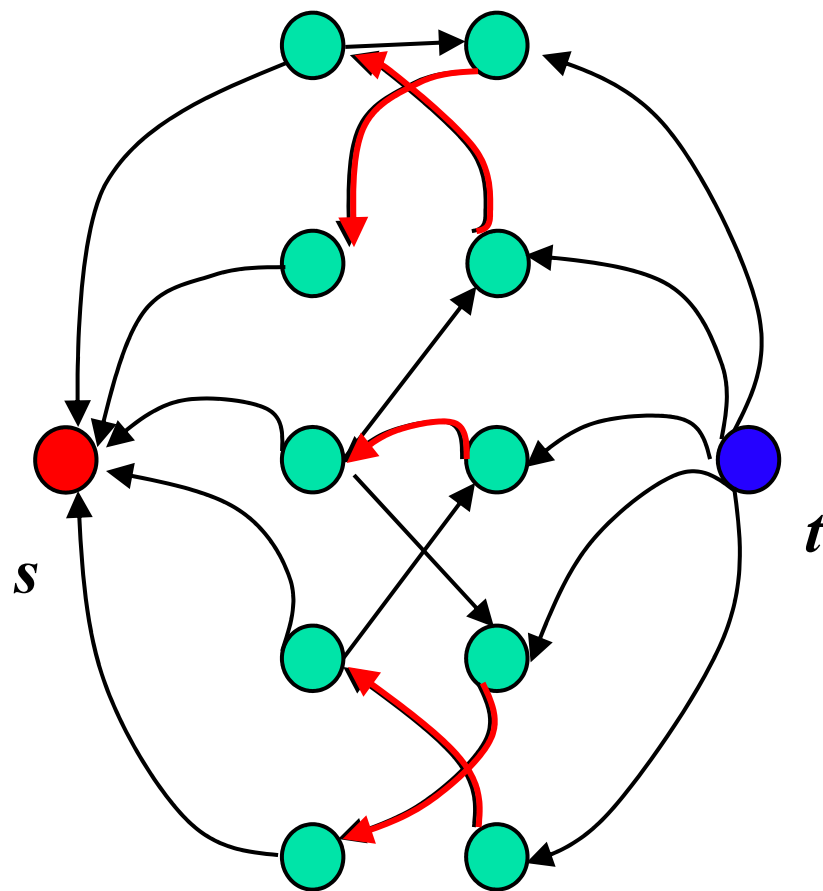


找不到更多的流，停止 115

兔子匹配—最大流算法

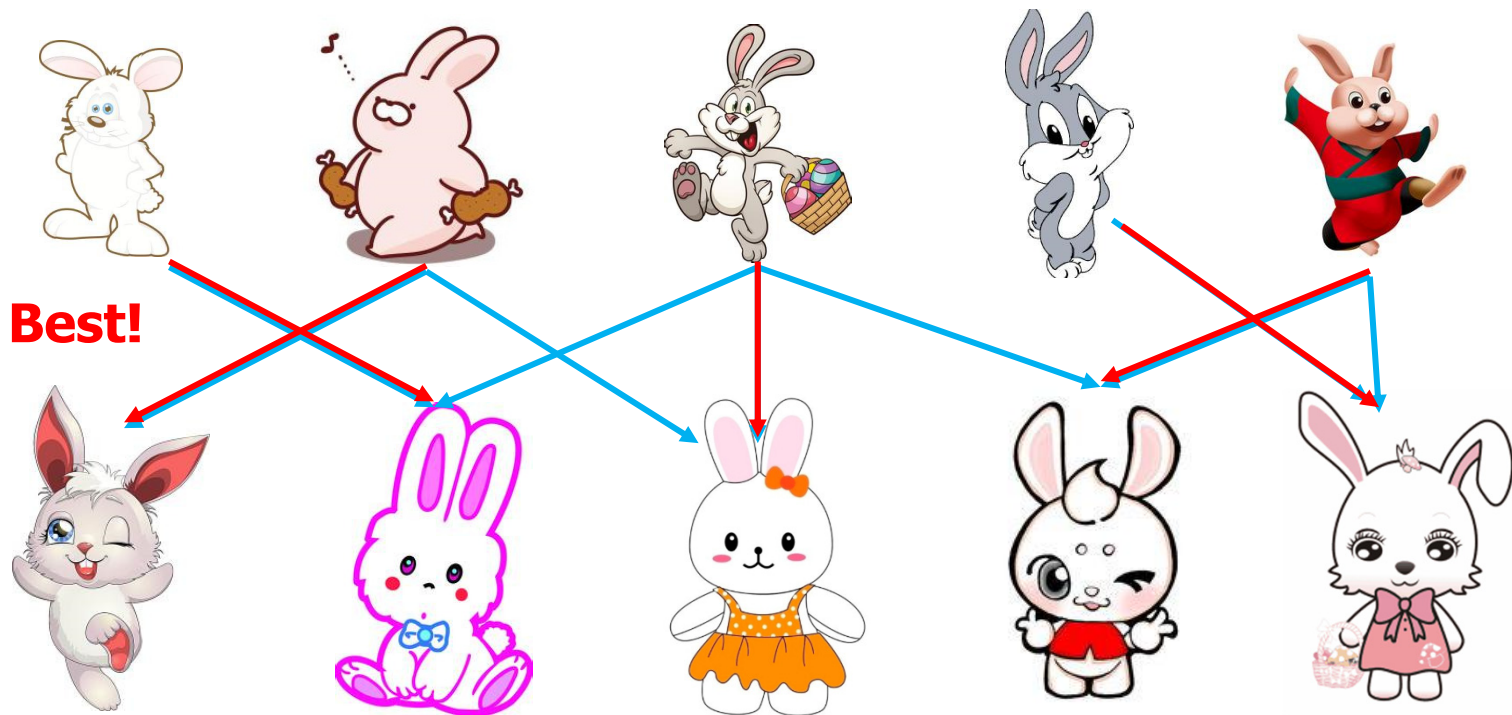
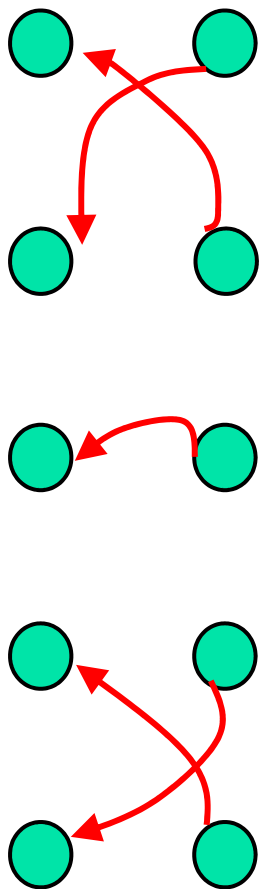
- 1) 找一条从 s 到 t 的流
- 2) 构建剩余网络
- 3) 在剩余网络中重复前两步，直到再也找不到从 s 到 t 的流
- 4) 构造问题的解
 - 若两顶点有反向边相连
 - 则这两点对应的兔子匹配

下图仅显示
有空余容量的边



构造问题的解

兔子匹配—问题的解



纸上得来终觉浅

绝知此事要躬行

推荐一本算法书

■ <https://jeffe.cs.illinois.edu/teaching/algorithms>

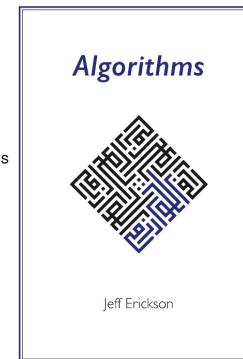
Algorithms

by Jeff Erickson

🔥 1st edition, June 2019 🔥
(Amazon links: [US](#), [UK](#), [DE](#), [ES](#), [FR](#), [IT](#), [JP](#))

This web page contains a free electronic version of my self-published textbook *Algorithms*, along with other lecture notes I have written for various theoretical computer science classes at the University of Illinois, Urbana-Champaign since 1998.

- [More information](#)
- [Get the book](#)
- [More algorithms lecture notes](#)
- [Models of computation notes](#)
- [Report an error](#) (separate page)
- [Coursework archive](#) (separate page)



◆ More Information ◆

Publication. A black-and-white paperback edition of the textbook can be purchased from [Amazon](#) for \$27.50. The full-color electronic version will remain freely available here indefinitely. (If there is enough demand, I may publish a full-color printed version of the *next* edition. Color printing is considerably more expensive; a full-color printed version of the current book would cost about \$75.)

Bug reports. After years of trying and failing to manage bug reports by email, I now maintain an issue-tracking page at [GitHub](#). If you find an error in the textbook, in the lecture notes, or in any other materials, [please submit a bug report](#). All other feedback is welcome as well.

Permissions. Anyone is welcome to download, print, use, copy, and/or distribute anything on this page, either electronically or on paper. You do not need to ask my permission, although I would appreciate hearing from you if you find this material useful. If you redistribute any of this material, please include a link back to [this web page](#), either directly or through the mnemonic shortcut <http://algorithms.wtf>. Specifically:

- The textbook *Algorithms* (in both paper and electronic forms) is licensed under a [Creative Commons Attribution 4.0 International license](#).
- All other lecture notes are licensed under a more restrictive [Attribution-NonCommercial-ShareAlike 4.0 International license](#).

Please do not ask me for solutions to the exercises. See [the course materials page](#) for an explanation.

Context. This material is the primary reference for two regularly-offered theoretical computer science courses at Illinois: CS 374 and CS 473. I taught these courses most recently in [Spring 2018](#) and [Spring 2017](#), respectively. I maintain a complete archive of [my past homeworks, exams, and lab handouts](#) on a separate page.

Prerequisites. The textbook assumes knowledge of discrete math (especially induction) and basic data structures and algorithms (especially recursion) consistent with the prerequisite courses CS 173 and CS 225 at Illinois. (See the [for more details](#).) [For a thorough overview of prerequisite material, I strongly recommend the following resources:](#)

- [Building Blocks for Theoretical Computer Science](#) by Margaret Fleck
- [Mathematics for Computer Science](#) by Eric Lehman, Tom Leighton, and Albert Meyer (I strongly recommend searching for the most recent revision.)